

دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق

درس رباتیک و بینایی ماشین

استاد: دکتر حمیدرضا تقی راد

استاد: دکتر محمدجواد احمدی

تمرین سری دوم

گروه آزادی

نام و نام خانوادگی	محمدامین محمدیون شبستری
شماره دانشجویی	۴۰۱۲۲۵۰۳
نام و نام خانوادگی	محمدسبحان سخایی
شماره دانشجویی	۴۰۱۱۹۲۵۳
نام و نام خانوادگی	هستی محمدی
شماره دانشجویی	۴۰۲۲۱۲۴۳
تاریخ	فروردین ۱۴۰۵



فهرست مطالب

۵	۱	سینماتیک مستقیم ربات KUKA iiwa
۵	۱.۱	استخراج DH
۵	۱.۱.۱	معرفی و بررسی قراردادهای دناویت-هارتنبِرگ (DH)
۶	۲.۱.۱	استخراج نهایی جدول DH بر اساس تصویر مسئله
۶	۲.۱	پایاده سازی سینماتیک مستقیم (Forward Kinematics Implementation)
۶	۱.۲.۱	کد
۱۲	۲.۲.۱	بررسی الگوریتم ها و ماتریس های تبدیل
۱۲	۳.۲.۱	تحلیل نتایج به دست آمده
۱۳	۴.۲.۱	نتیجه گیری
۱۳	۲	شبیه سازی سناریوی صنعتی سیستم بازرسی خودکار
۱۳	۱.۲	بخش A — پایاده سازی سیستم بینایی ماشین (OpenCV)
۱۴	۱.۱.۲	تسک A.1 — شبیه سازی و تولید تصویر میز کار
۱۶	۲.۱.۲	تسک A.2 — شناسایی چهار نشانگر گوشه میز
۲۲	۳.۱.۲	تسک A.3 — اصلاح پرسپکتیو تصویر (تبدیل به نمای عمودی)
۲۷	۴.۱.۲	تسک A.4 — بخش بندی و پیدا کردن دیسک ها در فضای HSV
۳۳	۵.۱.۲	تسک A.5 — تبدیل مختصات به دستگاه ربات و تکمیل نهایی پایپ لاین
۴۷	۲.۲	بخش B — سینماتیک مستقیم (Forward Kinematics) ربات PRR
۴۸	۱.۲.۲	تسک B.1 - جدول پارامترهای دناویت-هارتنبِرگ (DH)
۴۸	۲.۲.۲	تسک B.2 — محاسبات نمادین با SymPy
۵۱	۳.۲.۲	تسک B.3 — پایاده سازی عددی و اعتبارسنجی با Numpy
۵۳	۴.۲.۲	تسک B.4 — رسم فضای کاری ربات (Reachable Workspace)
۵۶	۵.۲.۲	تسک B.5 — رسیدن به قطعات با روش جستجوی سینماتیک مستقیم
۵۸	۳.۲	بخش D — سوالات تحلیلی
۵۸	۱.۳.۲	پاسخ سوال اول
۵۹	۲.۳.۲	پاسخ سوال دوم
۶۰	۳.۳.۲	پاسخ سوال سوم
۶۱	۴.۳.۲	سوال چهارم
۶۱	۵.۳.۲	سوال پنجم



فهرست تصاویر

۱۴	تصویر شبیه‌سازی شده (تصویر دوربین)	۱
۲۲	مشخص کردن نشانگرها روی تصویر به ترتیب ساعتگرد	۲
۲۷	مرکز نشانگرها در تصویر بعد از warp کردن پرسپکتیو	۴
۲۷	مرکز نشانگرها در تصویر قبل از warp کردن پرسپکتیو	۳
۳۲	مشخص کردن نشانگرها روی تصویر به ترتیب ساعتگرد	۵
۳۲	تصویرهای مستقل از دیسک‌ها با تفکیک بر اساس رنگ	۶
۵۵	نمودارهای فضای کاری ربات در حالت سه‌بعدی و نمای بالا	۷



فهرست جداول

۵	پارامترهای DH Standard (بر اساس مقالات مرجع)	۱
۶	پارامترهای Modified DH (استخراج شده از شکل مسئله)	۲
۱۳	مختصات نشانگرها در دستگاه مختصات پایه ربات	۳



فهرست برنامه‌ها

۱۸		کد تسک A.2
۲۴		کد تسک A.3
۲۸		کد تسک A.4
۳۵		کد تسک A.5
۴۸		کد تسک B.2
۵۱		کد تسک B.3
۵۳		کد تسک B.4
۵۶		کد تسک B.5

۱ سینماتیک مستقیم ربات KUKA iiwa

۱.۱ استخراج DH

۱.۱.۱ معرفی و بررسی قراردادهای دناویت-هارتنبرگ (DH)

برای استخراج سینماتیک مستقیم ربات‌ها، دو قرارداد اصلی برای تعیین دستگاه‌های مختصات و استخراج پارامترهای دناویت-هارتنبرگ وجود دارد: قرارداد استاندارد (Standard DH) و قرارداد اصلاح‌شده (Modified DH). در ادامه هر دو روش به طور کامل بررسی و مقایسه می‌شوند.

قرارداد استاندارد دناویت-هارتنبرگ (Standard DH)

این قرارداد که در سال ۱۹۵۵ معرفی شد، روش کلاسیک در سینماتیک رباتیک است. در این روش، دستگاه مختصات i در انتهای لینک i (روی مفصل $i + 1$) قرار می‌گیرد. در این حالت، محور z_{i-1} نشان‌دهنده راستای حرکت مفصل i است و محور x_i در امتداد عمود مشترک بین z_i و z_{i-1} تعریف می‌شود. ترتیب اعمال تبدیلات به صورت چرخش حول z_{i-1} ، انتقال در راستای z_{i-1} ، انتقال در راستای x_i و در نهایت چرخش حول x_i است.

در مقالات علمی چاپ شده در ژورنال‌های معتبر مانند SCIENCE DIRECT، پارامترهای استخراج شده برای بازوی رباتیک (مانند KUKA LBR iiwa) بر اساس قرارداد استاندارد معمولاً به شکل جدول ۱ ارائه می‌شوند.

جدول ۱: پارامترهای DH Standard (بر اساس مقالات مرجع)

مفصل (i)	α_i (رادیان)	a_i (متر)	d_i (متر)	θ_i (رادیان)
۱	$-\pi/2$	۰	d_1	θ_1
۲	$\pi/2$	۰	۰	θ_2
۳	$\pi/2$	۰	d_3	θ_3
۴	$-\pi/2$	۰	۰	θ_4
۵	$-\pi/2$	۰	d_5	θ_5
۶	$\pi/2$	۰	۰	θ_6
۷	۰	۰	d_7	θ_7

قرارداد اصلاح‌شده دناویت-هارتنبرگ (Modified DH)

این قرارداد توسط جان کریگ (John J. Craig) معرفی شد تا مشکلات روش استاندارد در ربات‌های دارای ساختار درختی را حل کند. در این روش، دستگاه مختصات i در ابتدای لینک i (روی خود مفصل i) قرار می‌گیرد. بنابراین، محور z_i مستقیماً نشان‌دهنده محور چرخش مفصل i است و محور x_i در امتداد عمود مشترک بین z_i و z_{i+1} قرار دارد. ترتیب تبدیلات در این روش با چرخش و انتقال حول راستای محور x_{i-1} شروع شده و با چرخش و انتقال حول راستای محور z_i پایان می‌یابد.



مقایسه و انتخاب قرارداد مبنا برای مسئله حاضر تفاوت اصلی این دو قرارداد در محل قرارگیری فریم‌ها است؛ روش استاندارد فریم را در انتهای لینک (Distal) و روش اصلاح‌شده آن را در ابتدای لینک (Proximal) قرار می‌دهد. این تفاوت باعث می‌شود که در روش اصلاح‌شده، محور z یک فریم دقیقاً منطبق بر مفصل متناظر خود باشد. با بررسی دقیق مختصات تعیین شده در شکل داده شده در صورت سوال، مشاهده می‌شود که دستگاه‌های مختصات دقیقاً منطبق بر قواعد قرارداد اصلاح‌شده (Modified DH) رسم شده‌اند. بنابراین، مبنای ما برای حل این سوال و استخراج پارامترها، جدول Standard DH ارائه شده در مقالات نیست، بلکه پارامترها باید بر اساس قواعد Modified DH استخراج شوند.

۲.۱.۱ استخراج نهایی جدول DH بر اساس تصویر مسئله

جدول پارامترهای DH اصلاح‌شده (مبنای حل) با توجه به دستگاه‌های مختصات رسم شده روی ربات که از قرارداد Modified DH پیروی می‌کنند، پارامترهای دناویت-هارتبرگ به صورت جدول ۲ استخراج می‌گردند.

جدول ۲: پارامترهای Modified DH (استخراج شده از شکل مسئله)

مفصل (i)	α_{i-1} (رادیان)	a_{i-1} (متر)	d_i (متر)	θ_i (رادیان)
۱	۰	۰	d_1	θ_1
۲	$-\pi/2$	۰	۰	θ_2
۳	$\pi/2$	۰	d_3	θ_3
۴	$\pi/2$	۰	۰	θ_4
۵	$-\pi/2$	۰	d_5	θ_5
۶	$-\pi/2$	۰	۰	θ_6
۷	$\pi/2$	۰	d_7	θ_7

۲.۱ پیاده‌سازی سینماتیک مستقیم (Forward Kinematics Implementation)

در این بخش، به منظور محاسبه موقعیت نهایی مجری نهایی (End-Effector) نسبت به پایه ربات، یک برنامه مستقل به زبان پایتون توسعه داده شده است. با توجه به وجود دو استاندارد رایج در استخراج پارامترهای دناویت-هارتبرگ، کد برنامه‌نویسی در دو حالت قرارداد اصلاح‌شده (Modified DH) و قرارداد استاندارد (Standard DH) پیاده‌سازی گردید تا صحت محاسبات ارزیابی شود. در زیر، کدهای پیاده‌سازی شده برای هر دو حالت آورده شده است:

۱.۲.۱ کد

```

۱ #!/usr/bin/env python
۲ # coding: utf-8
۳
۴ # # Implementation Code with Modified DH Table

```



```
5
6 # In[12]:
7
8
9 import numpy as np
10
11 def mdh_transformation_matrix(alpha, a, d, theta):
12     """
13     Calculates the homogeneous transformation matrix for a single link
14     using the Modified Denavit-Hartenberg (MDH / Craig's) convention.
15
16     Formula:  $T = Rot_x(\alpha) * Trans_x(a) * Rot_z(\theta) * Trans_z(d)$ 
17
18     Parameters:
19     alpha (float): Link twist angle in radians ( $\alpha_{i-1}$ )
20     a (float): Link length in meters ( $a_{i-1}$ )
21     d (float): Link offset in meters ( $d_i$ )
22     theta (float): Joint angle in radians ( $\theta_i$ )
23
24     Returns:
25     numpy.ndarray: 4x4 Homogeneous Transformation Matrix ( $T_{i-1}^i$ )
26     """
27     c_theta = np.cos(theta)
28     s_theta = np.sin(theta)
29     c_alpha = np.cos(alpha)
30     s_alpha = np.sin(alpha)
31
32     # MDH Transformation Matrix elements
33     T = np.array([
34         [c_theta,          -s_theta,          0,          a],
35         [s_theta * c_alpha,  c_theta * c_alpha, -s_alpha, -d * s_alpha],
36         [s_theta * s_alpha,  c_theta * s_alpha,  c_alpha,  d * c_alpha],
37         [0,                  0,                  0,                  1]
38     ])
39
40     return T
41
42 def forward_kinematics(thetas, d_params):
43     """
44     Calculates the forward kinematics of the 7-DOF robot using the MDH table.
```


*Parameters:**thetas (list or array): List of 7 joint angles [theta1, theta2, ..., theta7] in radians.**d_params (list or array): List of d offsets [d1, d3, d5, d7] in meters.**(d2, d4, d6 are zero based on the table).**Returns:**numpy.ndarray: Final 4x4 Transformation Matrix from Base (Frame 0) to End-Effector (Frame 7).**"""**# Unpack joint variables**t1, t2, t3, t4, t5, t6, t7 = thetas**# Unpack link offsets (d parameters)**d1, d3, d5, d7 = d_params**# Define the MDH table parameters: [alpha_{i-1}, a_{i-1}, d_i, theta_i]**mdh_table = [* *[0, 0, d1, t1], # Link 1* *[-np.pi/2, 0, 0, t2], # Link 2* *[np.pi/2, 0, d3, t3], # Link 3* *[np.pi/2, 0, 0, t4], # Link 4* *[-np.pi/2, 0, d5, t5], # Link 5* *[-np.pi/2, 0, 0, t6], # Link 6* *[np.pi/2, 0, d7, t7] # Link 7**]**# Initialize the total transformation matrix as an Identity matrix**T_0_to_7 = np.eye(4)**# Iteratively multiply the transformation matrices**for i in range(7):* *alpha, a, d, theta = mdh_table[i]* *# Get transformation matrix for the current link**T_i = mdh_transformation_matrix(alpha, a, d, theta)* *# Multiply with the cumulative transformation matrix* *# $T_0^n = T_0^1 * T_1^2 * \dots * T_{n-1}^n$* *T_0_to_7 = np.dot(T_0_to_7, T_i)**return T_0_to_7*



```
۸۵
۸۶ # =====
۸۷ # Example Usage and Testing
۸۸ # =====
۸۹ if __name__ == "__main__":
۹۰     # Example joint angles in radians (e.g., all joints at zero position)
۹۱     example_thetas = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
۹۲
۹۳     # Example link offsets in meters (replace with actual robot dimensions from datasheet)
۹۴     d1, d3, d5, d7 = 34.0, 40.0, 40.0, 126.0
۹۵     example_d_params = [34.0, 40.0, 40.0, 126.0]
۹۶
۹۷     # Calculate Forward Kinematics
۹۸     final_transformation = forward_kinematics(example_thetas, example_d_params)
۹۹
۱۰۰     # Extract position (X, Y, Z) and rotation matrix
۱۰۱     position = final_transformation[0:3, 3]
۱۰۲     rotation_matrix = final_transformation[0:3, 0:3]
۱۰۳
۱۰۴     print("Final Homogeneous Transformation Matrix (T07):")
۱۰۵     # Using np.round to clean up very small floating-point errors (like 1e-17)
۱۰۶     print(np.round(final_transformation, decimals=4))
۱۰۷
۱۰۸     print("\nEnd-Effector Position (X, Y, Z) in meters:")
۱۰۹     print(np.round(position, decimals=4))
۱۱۰
۱۱۱     print("--- Scenario 2: Joint 2 bent at 90 degrees ---")
۱۱۲     # Joint 2 with angle pi/2
۱۱۳     thetas_2 = [0, np.pi/2, 0, 0, 0, 0, 0]
۱۱۴     T2 = forward_kinematics(thetas_2, example_d_params)
۱۱۵     print("Expected Z:", d1, "| Expected X:", d3 + d5 + d7)
۱۱۶     print("Calculated Pos (X, Y, Z):", np.round(T2[0:3, 3], 4), "\n")
۱۱۷
۱۱۸     print("--- Scenario 3: Joint 4 bent at 90 degrees ---")
۱۱۹     # joint 4 with angle pi/2
۱۲۰     thetas_3 = [0, 0, 0, np.pi/2, 0, 0, 0]
۱۲۱     T3 = forward_kinematics(thetas_3, example_d_params)
۱۲۲     print("Expected Z:", d1 + d3, "| Expected X or Y extension:", d5 + d7)
۱۲۳     print("Calculated Pos (X, Y, Z):", np.round(T3[0:3, 3], 4), "\n")
۱۲۴
```



```
۱۲۵
۱۲۶ # # Implementation Code with Standard DH Table
۱۲۷
۱۲۸ # In[14]:
۱۲۹
۱۳۰
۱۳۱ import numpy as np
۱۳۲
۱۳۳ def dh_transform(theta, d, a, alpha):
۱۳۴     """Calculates the standard DH transformation matrix."""
۱۳۵     return np.array([
۱۳۶         [np.cos(theta), -np.sin(theta)*np.cos(alpha), np.sin(theta)*np.sin(alpha),
          ↪ a*np.cos(theta)],
۱۳۷         [np.sin(theta), np.cos(theta)*np.cos(alpha), -np.cos(theta)*np.sin(alpha),
          ↪ a*np.sin(theta)],
۱۳۸         [0, np.sin(alpha), np.cos(alpha), d],
۱۳۹         [0, 0, 0, 1]
۱۴۰     ])
۱۴۱
۱۴۲ def forward_kinematics_kuka_iwa(q, link_offsets):
۱۴۳     """
۱۴۴     Calculates the forward kinematics of the KUKA LBR iwa robot using Standard DH parameters.
۱۴۵     q: List of joint angles [theta1, ..., theta7] in radians.
۱۴۶     link_offsets: List of Z-axis offsets [d1, d3, d5, d7].
۱۴۷     """
۱۴۸     # Assign offsets to d variables
۱۴۹     d1, d3, d5, d7 = link_offsets
۱۵۰
۱۵۱     # DH table: [theta, d, a, alpha]
۱۵۲     dh_table = [
۱۵۳         [q[0], d1, 0, -np.pi/2],
۱۵۴         [q[1], 0, 0, np.pi/2],
۱۵۵         [q[2], d3, 0, np.pi/2],
۱۵۶         [q[3], 0, 0, -np.pi/2],
۱۵۷         [q[4], d5, 0, -np.pi/2],
۱۵۸         [q[5], 0, 0, np.pi/2],
۱۵۹         [q[6], d7, 0, 0]
۱۶۰     ]
۱۶۱
۱۶۲     # Initialize base transformation matrix as identity
```



```
۱۶۳ T_end_effector = np.eye(4)
۱۶۴
۱۶۵ # Multiply transformation matrices to find the final pose
۱۶۶ for params in dh_table:
۱۶۷     A_i = dh_transform(params[0], params[1], params[2], params[3])
۱۶۸     T_end_effector = np.dot(T_end_effector, A_i)
۱۶۹
۱۷۰ return T_end_effector
۱۷۱
۱۷۲ # --- Numerical test example ---
۱۷۳ if __name__ == "__main__":
۱۷۴     example_thetas = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
۱۷۵
۱۷۶     d1, d3, d5, d7 = 34.0, 40.0, 40.0, 126.0
۱۷۷     example_d_params = [d1, d3, d5, d7]
۱۷۸
۱۷۹     final_transformation = forward_kinematics_kuka_iiwa(example_thetas, example_d_params)
۱۸۰     position = final_transformation[0:3, 3]
۱۸۱
۱۸۲     print("Final Homogeneous Transformation Matrix (T_0^7):")
۱۸۳     print(np.round(final_transformation, decimals=4))
۱۸۴
۱۸۵     print("\nEnd-Effector Position (X, Y, Z) in meters:")
۱۸۶     print(np.round(position, decimals=4))
۱۸۷
۱۸۸     print("\n--- Scenario 2: Joint 2 bent at 90 degrees ---")
۱۸۹     thetas_2 = [0, np.pi/2, 0, 0, 0, 0, 0]
۱۹۰     T2 = forward_kinematics_kuka_iiwa(thetas_2, example_d_params)
۱۹۱     print("Expected Z:", d1, "| Expected X/Y:", d3 + d5 + d7)
۱۹۲     print("Calculated Pos (X, Y, Z):", np.round(T2[0:3, 3], 4))
۱۹۳
۱۹۴     print("\n--- Scenario 3: Joint 4 bent at 90 degrees ---")
۱۹۵     thetas_3 = [0, 0, 0, np.pi/2, 0, 0, 0]
۱۹۶     T3 = forward_kinematics_kuka_iiwa(thetas_3, example_d_params)
۱۹۷     print("Expected Z:", d1 + d3, "| Expected X/Y:", d5 + d7)
۱۹۸     print("Calculated Pos (X, Y, Z):", np.round(T3[0:3, 3], 4))
۱۹۹
```

۲.۲.۱ بررسی الگوریتم‌ها و ماتریس‌های تبدیل

همان‌طور که در کدهای بالا مشاهده می‌شود، دو تابع مجزا برای تولید ماتریس‌های تبدیل همگن (Homogeneous Transformation Matrices) نوشته شده است. تفاوت اصلی این دو پیاده‌سازی، در ترتیب ضرب ماتریس‌های پایه و در نتیجه قرارداد (Convention) استفاده شده است.

حالت اول: پیاده‌سازی بر اساس قرارداد اصلاح‌شده (Modified DH)

این حالت که بر اساس استاندارد جان کریگ (John J. Craig) است، مختصات فریم i را به مفصل i متصل می‌کند. در تابع `mdh_transformation_matrix` ترتیب ضرب ماتریس‌های انتقال و دوران به شکل زیر است:

$$T_i^{i-1} = Rot_x(\alpha_{i-1}) \cdot Trans_x(a_{i-1}) \cdot Rot_z(\theta_i) \cdot Trans_z(d_i)$$

با ضرب این چهار ماتریس، ماتریس تبدیل کلی در استاندارد MDH به دست می‌آید که در کد پیاده‌سازی شده است:

$$T_{MDH} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i & 0 & a_{i-1} \\ \sin \theta_i \cos \alpha_{i-1} & \cos \theta_i \cos \alpha_{i-1} & -\sin \alpha_{i-1} & -d_i \sin \alpha_{i-1} \\ \sin \theta_i \sin \alpha_{i-1} & \cos \theta_i \sin \alpha_{i-1} & \cos \alpha_{i-1} & d_i \cos \alpha_{i-1} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

حالت دوم: پیاده‌سازی بر اساس قرارداد استاندارد (Standard DH)

این حالت که به قرارداد پاول (Paul's Convention) نیز معروف است، فریم مختصات را به انتهای لینک متصل می‌کند. در تابع `dh_transform`، ترتیب عملیات ریاضی متفاوت بوده و به صورت زیر تعریف می‌شود:

$$T_i^{i-1} = Rot_z(\theta_i) \cdot Trans_z(d_i) \cdot Trans_x(a_i) \cdot Rot_x(\alpha_{i-1})$$

که منجر به تولید ماتریس تبدیل زیر می‌گردد (تفاوت از حالت قبل):

$$T_{Standard} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

۳.۲.۱ تحلیل نتایج به دست آمده

پس از اجرای اسکریپت‌ها بر روی سناریوهای تستی مختلف (حالت تمامی چرخش‌ها= صفر، چرخش ۹۰ درجه مفصل ۲ و چرخش ۹۰ درجه مفصل ۴)، مشاهده می‌کنیم که خروجی‌های عددی و موقعیت نهایی (X, Y, Z) در هر دو پیاده‌سازی کاملاً یکسان است. به عنوان مثال در حالت تمامی چرخش‌ها= صفر برای هر دو الگوریتم داریم:

$$P_{end-effector} = \begin{bmatrix} 0.0 \\ 0.0 \\ 1.266 \end{bmatrix}$$



و برای سناریوی دوم (انحراف مفصل دوم):

$$P_{end-effector} = \begin{bmatrix} 0.926 \\ 0.0 \\ 0.34 \end{bmatrix}$$

۴.۲.۱ نتیجه‌گیری

این تطابق دقیق نتایج، نشان‌دهنده صحت پیاده‌سازی هر دو الگوریتم است. اگرچه استانداردهای Modified DH و Standard DH در نحوه جایگذاری دستگاه‌های مختصات میانی (Intermediate Frames) روی مفاصل و همچنین در فرمولاسیون ماتریس‌های انتقال با یکدیگر تفاوت ساختاری دارند، اما از آنجا که هر دو ساختار فیزیکی و سینماتیکی یکتای ربات (KUKA LBR iiwa) را توصیف می‌کنند، ضرب ماتریس‌های آن‌ها از پایه تا مجری نهایی، در نهایت به یک موقعیت فضایی واحد و یکسان برای End-Effector ختم می‌شود.

۲ شبیه‌سازی سناریوی صنعتی سیستم بازرسی خودکار

۱.۲ بخش A — پیاده‌سازی سیستم بینایی ماشین (OpenCV)

دوربینی که در بالای میز کار ربات نصب شده، مستقیماً و عمود به پایین نگاه نمی‌کند، بلکه زاویه‌دار است. به همین دلیل، تصویری که به دست ما می‌رسد دچار اعوجاج پرسپکتیو (Projective / Keystone Distortion) شده و کمی هم نویز ملایم گوسی (Gaussian Noise) دارد. روی این میز کار، چهار نشانگر مربعی و مشکی‌رنگ در گوشه‌ها قرار داده شده‌اند که یک فضای کاری مربعی به ابعاد دقیق 0.6×0.6 متر را مشخص می‌کنند. در داخل این فضا، چند دیسک رنگی به عنوان قطعات هدف قرار گرفته‌اند.

قرارداد مختصات در این مسئله

اگر از گوشه بالا و سمت چپ میز کار فیزیکی شروع کنیم و در جهت عقربه‌های ساعت بچرخیم، موقعیت این چهار نشانگر مشکی در دستگاه مختصات پایه ربات به صورت زیر تعریف می‌شود (همه واحدها بر حسب متر هستند):

جدول ۳: مختصات نشانگرها در دستگاه مختصات پایه ربات

(+0.30, +0.30)	بالا-چپ	TL
(+0.30, -0.30)	بالا-راست	TR
(-0.30, -0.30)	پایین-راست	BR
(+0.30, +0.30)	پایین-چپ	BL

فرض کنید پایه ربات دقیقاً در نقطه مرکزی و زیر میز کار محکم شده است. همچنین سطح میز کار، صفحه $z = 0$ در مختصات جهانی در نظر گرفته شود.

۱.۱.۲ تسک A.1 — شبیه‌سازی و تولید تصویر میز کار

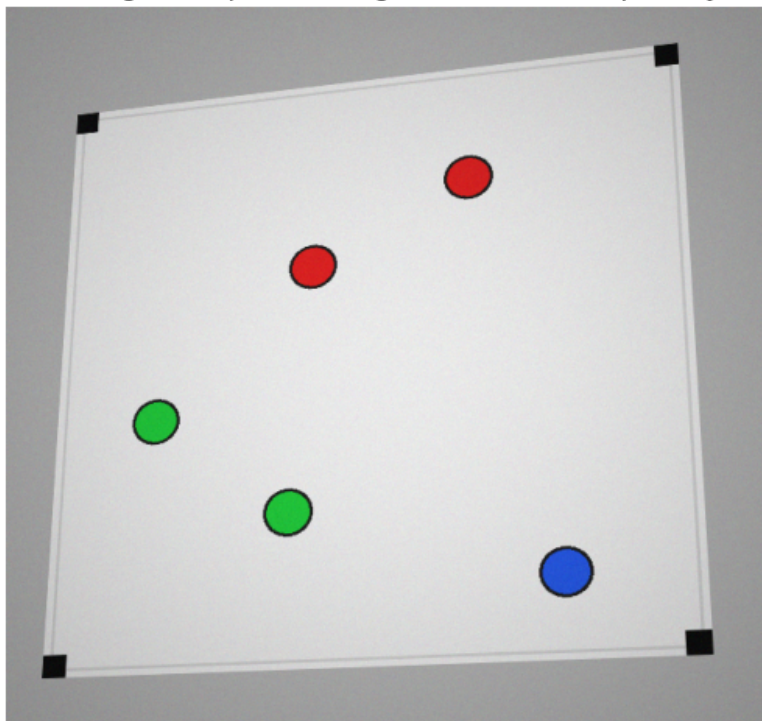
در محیط واقعی صنعتی، تصویر مستقیماً از دوربین گرفته می‌شود، اما در این تمرین تصویر دوربین با استفاده از کد داده شده شبیه‌سازی شده است. این کد به دست نخورده باقی مانده و تنها کتابخانه‌های لازم برای اجرای آن در سه خط ابتدایی اضافه شده‌اند.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

تصویر تولید شده (شکل ۱) شامل موارد زیر است:

- چهار مربع مشکی در گوشه‌ها که برای کالیبره کردن استفاده می‌شوند.
- پنج دیسک رنگی که مختصات واقعی و دقیق آن‌ها در متغیر GROUND_TRUTH_DISCS ذخیره شده است.
- شبیه‌سازی زاویه دوربین، اثر تاریکی گوشه‌های لنز (Vignetting) و کمی نویز طبیعی تصویر.

Raw camera image (oblique view, vignette, noise) — input to your pipeline



شکل ۱: تصویر شبیه‌سازی شده (تصویر دوربین)

خطاهای رایج در تصویربرداری

در یک سامانه بینایی ماشین کیفیت داده ورودی نقش تعیین کننده‌ای در عملکرد کل پایپ‌لاین پردازش تصویر دارد. تصویر ثبت شده توسط یک دوربین معمولی هرگز ایده‌آل نیست و معمولاً شامل انواع مختلفی از اعوجاج‌ها و خطاها است که در تصویر ورودی این مسئله شامل اعوجاج پرسپکتیو، نویز گاوسی و اثر تاریکی گوشه‌های لنز است.

وجود این خطاها می‌تواند مراحل بعدی پردازش تصویر مانند مشخص کردن گوشه‌ها، استخراج نشانگرها، تخمین ماتریس هموگرافی و تبدیل مختصات تصویر به مختصات واقعی را با خطا مواجه کند. در نتیجه اگر این خطاها در همان ابتدا مدل‌سازی و اصلاح نشوند، در مراحل بعدی تقویت شده و منجر به پدیده GIGO یا Garbage In, Garbage Out می‌شوند که به معنای نتیجه نامطلوب در اثر ورودی نامناسب است. به همین دلیل در طراحی پایپ‌لاین، معمولاً مرحله‌ای برای پیش‌پردازش تصویر در نظر گرفته می‌شود که شامل اصلاح خطاها است و هدف آن تولید داده‌ای پایدار و قابل اعتماد برای مراحل بعدی است.

اعوجاج پرسپکتیو (Keystone Distortion):

اگر در هنگام تصویربرداری از صحنه مورد نظر، دوربین کاملاً عمود قرار نگرفته باشد، تصویر ثبت شده دچار اعوجاج پرسپکتیو می‌شود و خطوطی که در فضای واقعی موازی هستند ممکن است در تصویر همگرا به نظر برسند. بر فرض مثال در این مسئله، به علت مایل بودن دوربین نسبت به صفحه میز، مربع‌های کالیبراسیون که در واقعیت شکل کاملاً مربعی دارند، در تصویر به صورت ذوزنقه یا متوازی‌الاضلاع دیده می‌شوند.

این خطا با تخمین ماتریس هموگرافی از روی نقاط متناظر، قابل اصلاح است و تصویر به نمایی معادل با دید عمود بر میز تبدیل می‌شود که این عمل اصلاح پرسپکتیو (Perspective Rectification) نام دارد و یکی از اجزای کلیدی در تبدیل مختصات تصویر به مختصات دنیای واقعی است.

نویز گاوسی (Gaussian Noise):

یکی دیگر از مشکلات رایج در تصاویر ثبت شده دوربین نویز سنسور است. سنسورهای تصویری به دلیل محدودیت‌های الکترونیکی و حرارتی دچار نوسانات تصادفی در اندازه‌گیری شدت نور می‌شوند. این نویز باعث می‌شود مقدار شدت روشنایی پیکسل‌ها به طور تصادفی تغییر کند. در نتیجه لبه‌ها و مرزهای اشیاء ممکن است کمی محو شده و آشکارسازی گوشه یا لبه دچار خطا شوند. به همین دلیل معمولاً پیش از استخراج داده‌ها و ویژگی‌های لازم از یک فیلتر برای کاهش اثر آن استفاده می‌شود. نویز ناشی از سنسورهای تصویری دوربین به طور معمول با یک توزیع گاوسی مدل‌سازی شده و نویز گاوسی نامیده می‌شود. مدل ریاضی متداول برای نویز گاوسی به صورت زیر است

$$I_{measured}(x, y) = I_{true}(x, y) + n(x, y)$$

که در آن I_{true} تصویر واقعی و $I_{measured}$ تصویر ثبت شده توسط دوربین است. عبارت $n(x, y)$ نیز به شکل زیر تعریف شده

$$n(x, y) \sim \mathcal{N}(0, \sigma^2)$$

و نویز گاوسی با توزیع σ را نشان می‌دهد که مقدار آن میزان شدت نویز را تعیین می‌کند. یکی از رایج‌ترین روش‌ها استفاده از فیلتر گاوسی است که با میانگین‌گیری وزن‌دار در یک همسایگی محلی، نوسانات فرکانس بالا را کاهش می‌دهد.

**اثر تاریکی گوشه‌های لنز (Vignetting):**

اثر تاریکی گوشه‌های لنز، یکی دیگر از خطاهای رایج در سیستم‌های تصویربرداری است که در آن روشنایی تصویر در نواحی پیرامونی کمتر از مرکز تصویر است. این پدیده معمولاً به دلیل طراحی اپتیکی لنز، محدودیت‌های مکانیکی یا افت شدت نور در مسیر اپتیکی ایجاد می‌شود.

این تغییر تدریجی در روشنایی می‌تواند الگوریتم‌هایی را که بر اساس آستانه‌گذاری یا مقایسه شدت روشنایی کار می‌کنند دچار مشکل کند. برای مثال در این مسئله، یک یک نشانگر سیاه در مرکز تصویر ممکن است به راحتی از پس زمینه تمایز داده شود، اما همان نشانگر در گوشه تصویر به دلیل تاریک‌تر بودن پس زمینه ممکن است به درستی تشخیص داده نشود.

یک مدل ساده برای این پدیده تعریف یک تابع وابسته به فاصله از مرکز تصویر است که در تصویر واقعی ضرب می‌شود. این تابع معمولاً به صورت شعاعی تعریف شده و با افزایش فاصله از مرکز تصویر کاهش می‌یابد و در رابطه زیر با $V(x, y)$ نمایش داده شده است:

$$I_{measured}(x, y) = V(x, y) \cdot I_{true}(x, y)$$

۲.۱.۲ تسک A.2 — شناسایی چهار نشانگر گوشه میز

هدف این قسمت نوشتن برنامه‌ای است که بتواند چهار مربع مشکی (نشانگرهای کالیبراسیون) را پیدا کند. خروجی باید آرایه‌ای از مختصات این چهار نقطه باشد که حتماً به ترتیب ساعت‌گرد و شروع از گوشه بالا-چپ تصویر (TL, TR, BR, BL) مرتب شده باشند. این ترتیب‌بندی برای هماهنگی با آرایه موقعیت جهانی در مراحل بعدی الزامی است.

برای تعیین نواحی نشانگرهای مربعی مشکی به صورت پایدار و قابل اعتماد، انتخاب نحوه نمایش تصویر و تعریف درست آستانه‌ها الزامی است و در مراحل بعدی پایپ‌لاین نقش اساسی و تعیین‌کننده‌ای دارد. بنابراین برای پاسخ به این سوال لازم است:

۱. فضای رنگی مناسب انتخاب شود.

۲. مقدار مناسب آستانه بر روی کانال درست از فضای تصویر اعمال شود تا نواحی مشکی نشانگرها از پس زمینه جداسازی شوند.

۳. کانتورها از تصویر باینری استخراج شوند.

۴. مرزها و لبه‌ها آشکار و مشخص شوند.

فضاهای رنگی (Color Spaces) و تاثیر آن‌ها بر پردازش تصویر

فضاهای رنگی، نحوه سازماندهی اطلاعات رنگی یک تصویر را مشخص می‌کنند. با انتخاب فضای مناسب برای تصویر قبل از شروع فرآیند، می‌توان سرعت پردازش و پایداری عملکرد در شرایط نوری متغیر را افزایش داد. چند فضای رنگی رایج عبارت‌اند از:

RGB: تصویر خامی که از دوربین دریافت می‌شود، معمولاً در فضای رنگی RGB نمایش داده می‌شود که در آن، هر پیکسل با سه مؤلفه رنگی توصیف می‌شود؛ یعنی برای هر پیکسل، سه مقدار معین بین 0 تا 255 نشان‌دهنده شدت رنگ‌های قرمز، سبز و آبی هستند. از نظر هندسی اگر در مختصات کارتزین (Cartesian Coordinates) سه محور x - y - z را به ترتیب شدت رنگ‌های قرمز، سبز و آبی فرض کنیم، فضای RGB در داخل یک مکعب با ضلع 256 قرار می‌گیرد و دو راس از این مکعب به مختصات‌های $(0, 0, 0)$ و $(255, 255, 255)$ به ترتیب نشان‌دهنده مشکی و سفید هستند.

بنابراین در این فضا، شدت روشنایی و اطلاعات رنگی درهم آمیخته شده‌اند و در نتیجه تصویر رنگی در فضای RGB نسبت به تغییرات در روشنایی و تاریکی بسیار حساس هستند. این مسئله باعث می‌شود که این نمایش برای برخی از اهداف در بینایی ماشین انتخاب مناسبی نباشد.

HSV (Hue, Saturation, Value): در فضای HSV هر پیکسل با سه مؤلفه رنگ (Hue)، شدت رنگ (Saturation) و شدت نور (Value) مشخص می‌شود. جداسازی رنگ از شدت روشنایی باعث می‌شود که این فضا نسبت به RGB بیشتر مورد استفاده قرار بگیرد. از نظر هندسی، این فضا همان مدل RGB را در مختصات استوانه‌ای نمایش می‌دهد؛ به طوری که سه مؤلفه (r, θ, z) معادل (S, H, V) می‌باشند.

Grayscale: در فضای Grayscale، تمام اطلاعات رنگی حذف شده و تنها شدت روشنایی به صورت یک تابع اسکالر نگه داشته می‌شود. در نتیجه در زمان‌هایی که به دنبال مشخص کردن نواحی خیلی تیره یا خیلی روشن هستیم، استفاده از این فضا می‌تواند سرعت پردازش را به حداکثر برساند.

LAB (CIELAB): فضای LAB کل بازه رنگ‌هایی که انسان توانایی دیدنشان را دارد را در بر گرفته و به علت جداسازی شدت روشنایی از اطلاعات رنگی، پایدارترین عملکرد را در محیط طبیعی با شدت روشنایی‌های متغیر دارد. مؤلفه L شدت روشنایی، A شدت رنگ‌های متضاد سبز (سمت منفی) و قرمز (سمت مثبت) و B شدت رنگ‌های متضاد آبی (در سمت منفی) و زرد (در سمت مثبت) را نشان می‌دهند.

تعریف آستانه (Threshold)

با تعریف آستانه روی کانال‌های فضای رنگی، می‌توان تصویر را به یک تصویر باینری تبدیل کرد، به طوری که پیکسل‌هایی که از یک شرط مشخص پیروی می‌کنند در یک گروه و بقیه در گروهی دیگر قرار داده می‌شوند. آستانه‌گذاری به دو روش سراسری و تطبیقی انجام می‌شود. در آستانه‌گذاری سراسری یک مقدار آستانه ثابت برای کل تصویر انتخاب می‌شود و اگر فرض شود که آستانه T روی یک کانال از فضای رنگی تعریف شده و $I(x, y)$ مقدار آن مؤلفه در یک پیکسل دلخواه را نشان می‌دهد، می‌توان نوشت:

$$B(x, y) = \begin{cases} 1 & I(x, y) < T \\ 0 & I(x, y) \geq T \end{cases}$$

این روش با وجود ساده بودن، نسبت به تغییرات روشنایی حساس است. در آستانه‌گذاری تطبیقی، برای هر ناحیه از تصویر آستانه بر اساس همسایگی محلی تعیین شده و تابعی از مکان تصویر است. به صورت ریاضی می‌توان نوشت:

$$B(x, y) = \begin{cases} 1 & I(x, y) < T(x, y) \\ 0 & I(x, y) \geq T(x, y) \end{cases}$$

که در آن $T(x, y)$ معمولاً بر اساس میانگین وزن‌دار مؤلفه مشخص شده در همسایگی پیکسل محاسبه می‌شود. این روش در حضور vignetting، سایه‌ها و روشنایی‌های ناهمگن پایدارتر است.

**کانتور (Contour) و لبه (Edge)**

پس از باینری کردن تصویر، نواحی متصل مربوط به اشیاء قابل استخراج هستند. مجموعه نقاطی که در آن‌ها تغییر شدت روشنایی زیاد است را لبه می‌نامند. لبه یک مفهوم گرادیانی است که معمولاً روی اندازه گرادیان شدت روشنایی تعریف می‌شود. اگر $I(x, y)$ شدت روشنایی باشد، گرادیان به صورت زیر تعریف می‌شود:

$$\nabla I(x, y) = \begin{bmatrix} \frac{\partial I}{\partial x} \\ \frac{\partial I}{\partial y} \end{bmatrix}$$

اندازه گرادیان نیز با

$$\|\nabla I(x, y)\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$$

برابر است و نشان می‌دهد که شدت روشنایی در تصویر با چه سرعتی تغییر می‌کند. نقاطی که این مقدار در آن‌ها بزرگ است، کاندیدهای لبه هستند.

در مقابل، کانتور معمولاً مرز هندسی بسته پیرامون ناحیه یک شیء است. در یک تصویر باینری شده کانتور مرز میان نواحی با مقدار ۰ و ۱ را نشان می‌دهد و برای توصیف شکل جسم، محاسبه مساحت، محیط و مرکز هندسی مناسب‌تر است.

کد پیاده‌سازی شده و توضیحات آن

```

1 import cv2
2 import numpy as np
3
4 def detect_corner_markers(image):
5     """
6     Find the 4 black square calibration markers.
7     Returns: corners array sorted [TL, TR, BR, BL]
8     """
9
10    # bgr --> hsv
11    hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
12
13    # threshold
14    _, binary = cv2.threshold(
15        hsv[:, :, 2],
16        50,
17        255,
18        cv2.THRESH_BINARY_INV
19    )
20
```



```
۲۱     # external contours
۲۲     contours, _ = cv2.findContours(
۲۳         binary,
۲۴         cv2.RETR_EXTERNAL,
۲۵         cv2.CHAIN_APPROX_SIMPLE
۲۶     )
۲۷
۲۸     # filter by size & squareness
۲۹     square_candidates = []
۳۰
۳۱     for cnt in contours:
۳۲         area = cv2.contourArea(cnt)
۳۳
۳۴         if area < 100:
۳۵             continue
۳۶
۳۷         x, y, w, h = cv2.boundingRect(cnt)
۳۸
۳۹         box_area = w * h
۴۰         squareness = area / box_area if box_area > 0 else 0
۴۱
۴۲         if squareness > 0.3:
۴۳             square_candidates.append(cnt)
۴۴
۴۵     # largest 4
۴۶     square_candidates.sort(key=cv2.contourArea, reverse=True)
۴۷     top4 = square_candidates[:4]
۴۸
۴۹     if len(top4) < 4:
۵۰         raise ValueError("Could not detect 4 corner markers")
۵۱
۵۲     # centroids
۵۳     centers = []
۵۴
۵۵     for cnt in top4:
۵۶         M = cv2.moments(cnt)
۵۷
۵۸         if M['m00'] == 0:
۵۹             continue
۶۰
```



```
۶۱     cx = int(M['m10'] / M['m00'])
۶۲     cy = int(M['m01'] / M['m00'])
۶۳
۶۴     centers.append((cx, cy))
۶۵
۶۶     # sort
۶۷     centers = sort_clockwise(centers)
۶۸
۶۹     return np.float32(centers)
۷۰
۷۱
۷۲ def sort_clockwise(pts):
۷۳
۷۴     pts = sorted(pts, key=lambda p: p[1])
۷۵
۷۶     top = sorted(pts[:2], key=lambda p: p[0])
۷۷     bottom = sorted(pts[2:], key=lambda p: p[0], reverse=True)
۷۸
۷۹     return [top[0], top[1], bottom[0], bottom[1]]
۸۰
۸۱
۸۲ # load image
۸۳ image_path = r"C:\Users\Asus\Desktop\hasti\output.png" # change to your image path
۸۴ image = cv2.imread(image_path)
۸۵
۸۶ if image is None:
۸۷     raise FileNotFoundError(f"Image not found: {image_path}")
۸۸
۸۹ # detect markers
۹۰ corners = detect_corner_markers(image)
۹۱
۹۲ print("Detected corners:")
۹۳ print(corners)
۹۴
۹۵ # draw detected points
۹۶ output = image.copy()
۹۷
۹۸ labels = ["TL", "TR", "BR", "BL"]
۹۹
۱۰۰ for i, (x, y) in enumerate(corners.astype(int)):
```

```

۱۰۱ cv2.circle(output, (x, y), 10, (0, 0, 255), -1)
۱۰۲
۱۰۳ cv2.putText(
۱۰۴     output,
۱۰۵     labels[i],
۱۰۶     (x + 10, y - 10),
۱۰۷     cv2.FONT_HERSHEY_SIMPLEX,
۱۰۸     0.7,
۱۰۹     (0, 255, 0),
۱۱۰     2
۱۱۱ )
۱۱۲
۱۱۳ # save & show
۱۱۴ cv2.imwrite("detected_markers.jpg", output)
۱۱۵ cv2.imshow("Detected Markers", output)
۱۱۶ cv2.waitKey(0)
۱۱۷ cv2.destroyAllWindows()
۱۱۸
۱۱۹
    
```

در مسئله فعلی هدف تشخیص نشانگرهای سیاه است، بنابراین فضاهاى رنگی که در آن روشنایی از رنگ متمایز شده (مانند Grayscale یا HSV) انتخاب مناسب‌تری هستند؛ چرا که وجه تمایز این نشانگرها نسبت به زمینه تیره بودن آن‌ها است نه رنگ. علاوه بر این می‌دانیم روشنایی تصویر یکنواخت نیست و اثر تاریکی گوشه‌های لنز وجود دارد.

بنابراین دو راه پیش رو، تبدیل تصویر به Grayscale و استفاده از آستانه‌گذاری تطبیقی یا تبدیل تصویر به HSV و تعریف یک آستانه برای کانال روشنایی (Value) هستند. روش اول حجم مسئله و داده‌ها را کاهش می‌دهد، با این حال در روش دوم، اعمال آستانه سراسری کافی است و به علاوه در تسک A.4 برای پیدا کردن دیسک‌های رنگی، دیگر نیازی به تبدیل دوباره تصویر به HSV نیست. در نتیجه عکس را در ابتدا به HSV تبدیل می‌کنیم. در این فضا Value رنگ مشکی صفر است، پس Value پیکسل‌ها باید از حد آستانه انتخاب شده برای باینری کردن (Binarization) تصویر کوچکتر باشند.

در تعیین مقدار آستانه نه باید آنقدر سخت‌گیرانه عمل کنیم که نشانگرها در تصویر باینری غیرقابل تشخیص شوند و نه باید آنقدر محدوده را باز بگذاریم که تاریکی‌ها و سایه‌های تصویر اجازه تشخیص درست نشانگرها را از ما بگیرند. بدین منظور، پس از کمی آزمون و خطا مقدار آستانه برای رنگ مشکی در کد 50 در نظر گرفته شده است.

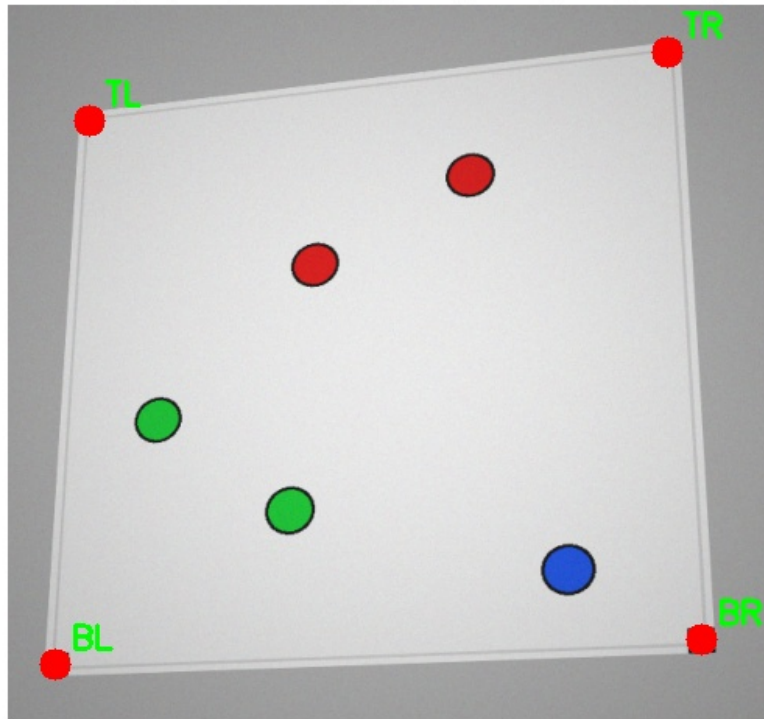
در مرحله بعدی به دنبال استخراج کانتورها می‌رویم که در اینجا کاربردی‌تر از اتکا به لبه‌هاست، زیرا می‌توانیم مرز بسته نشانگرهای مربعی تیره را پس از باینری کردن تصویر مستقیماً به صورت کانتور استخراج کنیم.

حال برای جدا کردن این ۴ نشانگر از دیگر نویزها، کانتورها را بر اساس مساحت و مربع بودن فیلتر می‌کنیم. برای این کار ابتدا مساحت هر کانتور را به دست آورده و اگر این مساحت کوچکتر از 100 باشد، کانتور متناظر از لیست کاندیدها حذف می‌شود. سپس مساحت کانتورهای باقی‌مانده را بر مساحت کوچکترین مستطیلی که محیط بر آن است (Bounding Rectangle) تقسیم می‌کنیم و اگر حاصل تقسیم از 0.3 بزرگتر باشد، به لیست کاندیدها اضافه می‌شود. در نهایت چهار کاندیدی که بیشترین مساحت را دارند انتخاب شده، مرکز آنها محاسبه می‌شود و به ترتیب ساعت‌گرد با شروع از گوشه بالا-چپ مرتب می‌شوند.

خروجی کد

تصویر خروجی این کد در شکل ۲ آمده است.

Raw camera image (oblique view, vignette, noise) — input to your pipeline



شکل ۲: مشخص کردن نشانگرها روی تصویر به ترتیب ساعتگرد

خروجی دیگر کد این بخش، مختصات پیکسلی چهار نشانگر است که به صورت زیر است:

Detected corners:

[[127. 106.]

[498. 62.]

[520. 439.]

[105. 455.]]

۳.۱.۲ تسک A.3 — اصلاح پرسپکتیو تصویر (تبدیل به نمای عمودی)

در این تسک، با داشتن مختصات پیکسلی چهار گوشه و مختصات واقعی آن‌ها روی میز کار، ماتریس هوموگرافی (Homography Matrix) را به دست آورده و با استفاده از آن تصویر مایل دوربین را به تصویر جدیدی به اندازه 600×600 پیکسل از نمای کاملاً بالا به پایین (Top-Down) تبدیل می‌کنیم.

برای تصویر جدید شرط‌های زیر وجود دارد:



۱. نقطه مرکزی میز کار باید دقیقاً در وسط تصویر یعنی پیکسل (300, 300) قرار بگیرد.
۲. هر ۱ متر در دنیای واقعی باید معادل با ۱۰۰۰ پیکسل در تصویر باشد (با این مقیاس، میز ۶۰ در ۶۰ سانتی متری دقیقاً کل ادر ۶۰۰ در ۶۰۰ پیکسلی را پر می کند).
۳. ضریب تبدیل پیکسل به متر باید در کد تعریف شود و برای اطمینان از صحت کار، یک شبکه شطرنجی (Grid) با فاصله های ۶ سانتی متری روی تصویر اصلاح شده رسم شود.

ماتریس هوموگرافی

از آن جا که میز یک صفحه در فضای سه بعدی است و تمام نشانگرها روی صفحه $z = 0$ قرار دارند، می توان رابطه ای مستقیم بین نقاط روی صفحه میز و پیکسل های تصویر دوربین برقرار کرد. همانطور که پیشتر نیز گفته شد، چون دوربین نسبت به میز زاویه دراد، مربع های روی میز در تصویر به شکل دوزنقه دیده شده و موازی ها در تصویر همگرا می شوند.

ماتریس هوموگرافی ابزاری است که این اعوجاج پرسپکتیو را مدل می کند؛ یعنی با داشتن چهار نقطه گوشه (در این مسئله چهار نشانگر TL, TR, BR, BL) می توان تبدیل را تخمین زد و سپس تصویر را طوری Warp کرد که صفحه میز مانند حالتی که دوربین دقیقاً عمود بر آن است مشاهده شود. این ماتریس یک تبدیل Projective بین دو صفحه است که در مختصات همگن نمایش داده می شود.

اگر یک نقطه در تصویر ورودی $\mathbf{p} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$ و نقطه متناظر آن در تصویر خروجی اصلاح شده $\mathbf{p}' = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$ باشد، داریم:

$$\mathbf{p}' \sim \mathbf{H}\mathbf{p}$$

که $\mathbf{H}$ یک ماتریس 3×3 است به شکل زیر است:

$$\mathbf{H} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

با بازنویسی رابطه، مختصات اقلیدسی خروجی از تقسیم بر مولفه سوم به دست می آید:

$$\begin{bmatrix} \tilde{x}' \\ \tilde{y}' \\ \tilde{w}' \end{bmatrix} = \mathbf{H} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad x' = \frac{\tilde{x}'}{\tilde{w}'}, \quad y' = \frac{\tilde{y}'}{\tilde{w}'}$$

از آن جا که $\mathbf{H}$ دارای ۹ مولفه است ولی فقط تا یک مقیاس معنا دارد، ۸ درجه آزادی دارد؛ لذا حداقل ۴ نقطه غیرهم خط متناظر برای حل آن لازم است. در این مسئله چهار نشانگر کالیبراسیون روی گوشه های میز چنین نقاطی را فراهم می کنند. به صورت کاربردی، اگر چهار گوشه میز در تصویر دوربین مشخص شوند و چهار نقطه مقصد در صفحه خروجی تعیین گردد، می توان تصویری ساخت که در آن صفحه میز مستطیلی و با مقیاس مطلوب نمایش داده شود.

کد پیاده سازی شده و توضیحات آن

برای پیاده سازی این قسمت کافی است:

۱. ابتدا یک تصویر خالی به اندازه 600×600 تولید کنیم.

۲. با استفاده از مختصات فعلی و مقصد نشانگرها و دستور `findHomography.cv2` ماتریس هوموگرافی را محاسبه می‌کنیم.

۳. در نهایت تصویر با استفاده از `warpPerspective.cv2` به نمای بالا به پایین تبدیل می‌شود.

برای به دست آوردن نمای کاملاً بالا به پایین و مرکز تصویر قبل و بعد از تصحیح پرسپکتیو، کد زیر نوشته شده است:

```
۱
۲ original_display = image.copy()
۳
۴ # center
۵ p1 = corners[0]
۶ p2 = corners[2]
۷
۸ p3 = corners[1]
۹ p4 = corners[3]
۱۰
۱۱ x1, y1 = p1.astype(np.float64)
۱۲ x2, y2 = p2.astype(np.float64)
۱۳
۱۴ x3, y3 = p3.astype(np.float64)
۱۵ x4, y4 = p4.astype(np.float64)
۱۶
۱۷ # p1 - p2
۱۸ A1 = y2 - y1
۱۹ B1 = x1 - x2
۲۰ C1 = A1 * x1 + B1 * y1
۲۱
۲۲ # p3 - p4
۲۳ A2 = y4 - y3
۲۴ B2 = x3 - x4
۲۵ C2 = A2 * x3 + B2 * y3
۲۶
۲۷ det = A1 * B2 - A2 * B1
۲۸
۲۹ if abs(det) < 1e-10:
۳۰     raise ValueError("Diagonal lines are parallel or ill-conditioned.")
۳۱
۳۲ center_x = (B2 * C1 - B1 * C2) / det
۳۳ center_y = (A1 * C2 - A2 * C1) / det
۳۴
```



```
۳۵ draw_center_x = int(round(center_x))
۳۶ draw_center_y = int(round(center_y))
۳۷
۳۸ old_center = np.array([[[center_x, center_y]]], dtype=np.float32)
۳۹
۴۰ cv2.circle(
۴۱     original_display,
۴۲     (draw_center_x, draw_center_y),
۴۳     4,
۴۴     (255, 0, 0),
۴۵     -1
۴۶ )
۴۷
۴۸ cv2.putText(
۴۹     original_display,
۵۰     "CENTER",
۵۱     (draw_center_x + 8, draw_center_y),
۵۲     cv2.FONT_HERSHEY_SIMPLEX,
۵۳     0.2,
۵۴     (255, 0, 0),
۵۵     1
۵۶ )
۵۷
۵۸ # rectification
۵۹ canvas_size = 600
۶۰
۶۱ dst_points = np.float32([
۶۲     [0, 0],
۶۳     [599, 0],
۶۴     [599, 599],
۶۵     [0, 599]
۶۶ ])
۶۷
۶۸ H, _ = cv2.findHomography(corners, dst_points)
۶۹
۷۰ rectified = cv2.warpPerspective(
۷۱     image,
۷۲     H,
۷۳     (canvas_size, canvas_size)
۷۴ )
```



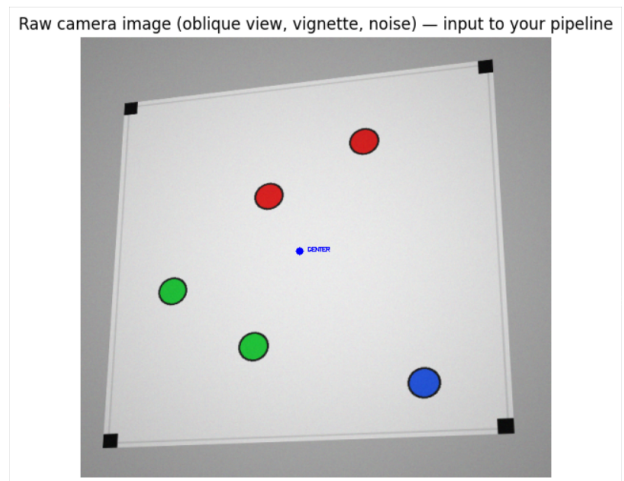
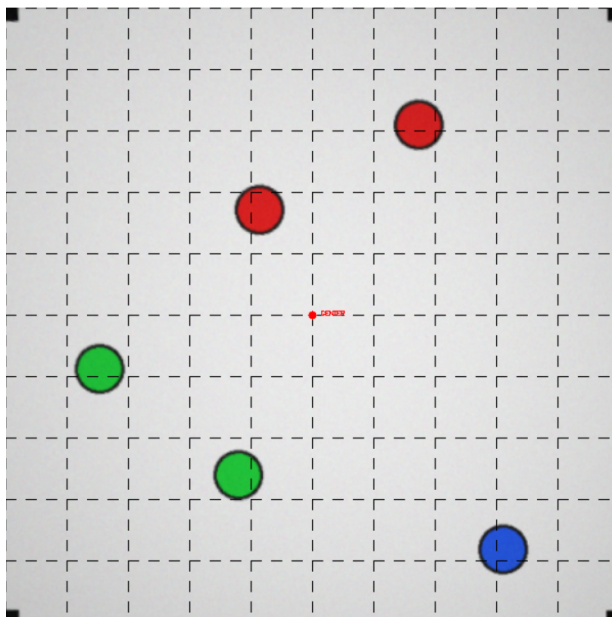
```
۷۵
۷۶ # grid
۷۷ spacing = 60
۷۸ dash_length = 10
۷۹ gap_length = 10
۸۰
۸۱ for x in range(0, canvas_size, spacing):
۸۲     y = 0
۸۳     while y < canvas_size:
۸۴         y_end = min(y + dash_length, canvas_size)
۸۵         cv2.line(rectified, (x, y), (x, y_end), (0, 0, 0), 1)
۸۶         y += dash_length + gap_length
۸۷
۸۸ for y in range(0, canvas_size, spacing):
۸۹     x = 0
۹۰     while x < canvas_size:
۹۱         x_end = min(x + dash_length, canvas_size)
۹۲         cv2.line(rectified, (x, y), (x_end, y), (0, 0, 0), 1)
۹۳         x += dash_length + gap_length
۹۴
۹۵ # new center
۹۶ new_center = cv2.perspectiveTransform(old_center, H)
۹۷
۹۸ new_x = int(round(new_center[0][0][0]))
۹۹ new_y = int(round(new_center[0][0][1]))
۱۰۰
۱۰۱ # smaller transformed center marker
۱۰۲ cv2.circle(
۱۰۳     rectified,
۱۰۴     (new_x, new_y),
۱۰۵     4,
۱۰۶     (0, 0, 255),
۱۰۷     -1
۱۰۸ )
۱۰۹
۱۱۰ cv2.putText(
۱۱۱     rectified,
۱۱۲     "CENTER",
۱۱۳     (new_x + 8, new_y),
۱۱۴     cv2.FONT_HERSHEY_SIMPLEX,
```

```

۱۱۵     0.2,
۱۱۶     (0, 0, 255),
۱۱۷     1
۱۱۸ )
۱۱۹
۱۲۰ # display
۱۲۱ cv2.imshow("Original Image - Center", original_display)
۱۲۲ cv2.imshow("Rectified Image - Grid + Transformed Center", rectified)
۱۲۳
۱۲۴ cv2.waitKey(0)
۱۲۵ cv2.destroyAllWindows()
۱۲۶
    
```

خروجی کد

خروجی‌های کد در شکل ۳ و ۴ قابل مشاهده‌اند.



شکل ۳: مرکز نشانگرها در تصویر قبل از warp کردن پرسپکتیو

شکل ۴: مرکز نشانگرها در تصویر بعد از warp کردن پرسپکتیو

۴.۱.۲ تسک A.4 — بخش‌بندی و پیدا کردن دیسک‌ها در فضای HSV

در این تسک، روی تصویر اصلاح شده مرحله قبل، دیسک‌های قرمز، آبی و سبز را پیدا کرده و همچنین مختصات مرکز (Centroid) هر کدام را استخراج می‌کنیم.

چالش‌های این مرحله

۱. رنگ قرمز در فضای HSV:

در فضای رنگی HSV، مولفه Hue در یک دایره ۳۶۰ درجه تعریف می‌شود. رنگ‌های آبی و سبز در نواحی میانی این دایره قرار دارند و با یک بازه پیوسته تعریف می‌شوند. اما رنگ قرمز در ابتدا و انتهای دایره قرار دارد (Wrap-around). بنابراین برای شناسایی کامل طیف قرمز، باید دو بازه مجزا تعریف کرده و حاصل‌های به دست آمده را با عملگر OR ترکیب کنیم.

۲. اثر تیرگی و سایه روشن لنز پس از اصلاح تصویر

اثر تیرگی گوشه‌های لنز باعث کاهش روشنایی در لبه‌های تصویر شده و سایه‌ها باعث تغییر فام (در فضای RGB) می‌گردند. استفاده از فضای HSV تا حدی مقاومت ایجاد می‌کند، اما کافی نیست. بنابراین برای حذف نویزهای کوچک با تعریف یک مساحت حداقلی، یک فیلتر روی مساحت اشیاء پیدا شده اعمال می‌کنیم.

۳. احتمال به هم چسبیده بودن دیسک‌های هم‌رنگ

ممکن است در تصویر دو دیسک هم‌رنگ، چسبیده به هم قرار گرفته باشند. بنابراین نباید کل پیکسل‌های یک رنگ، یک شیء واحد در نظر گرفته شوند. الگوریتم Connected-Component Analysis هر ناحیه پیوسته را برچسب Label می‌زند و برای تفکیک آن‌ها می‌توان از عملیات‌های مورفولوژیک cv2.morphologyEx استفاده کرد، تا هر دیسک به درستی و مجزا شمرده شود.

کد پیاده‌سازی شده

کد این بخش در قسمت زیر قابل مشاهده است:

```
1 import cv2
2 import numpy as np
3
4 def segment_discs_and_show(path_to_png):
5     """
6     Load HSV image from file, segment colored discs, separate touching components.
7     Show results and return list of (color_name, u, v) in pixel coordinates.
8     """
9     # read a3
10    rectified_img = cv2.imread(path_to_png, cv2.IMREAD_COLOR)
11    if rectified_img is None:
12        raise FileNotFoundError(f"Cannot load image: {path_to_png}")
13
14    hsv = cv2.cvtColor(rectified_img, cv2.COLOR_BGR2HSV)
15    results = []
16
17    # blue & green: 1 range/ red: 2 ranges
18    color_ranges = {
```



```
۱۹     'red': [  
۲۰         ([0, 100, 100], [10, 255, 255]),    # low red  
۲۱         ([170, 100, 100], [180, 255, 255])    # high red  
۲۲     ],  
۲۳     'green': [  
۲۴         ([40, 80, 80], [80, 255, 255])  
۲۵     ],  
۲۶     'blue': [  
۲۷         ([100, 80, 80], [130, 255, 255])  
۲۸     ],  
۲۹ }  
۳۰  
۳۱  
۳۲     # kernels morphological separation  
۳۳  
۳۴     open_kernel      = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))  
۳۵     close_kernel     = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))  
۳۶     separate_kernel  = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))  
۳۷  
۳۸     # copy of original  
۳۹     vis_img = rectified_img.copy()  
۴۰  
۴۱     # process each color  
۴۲     for color_name, ranges in color_ranges.items():  
۴۳  
۴۴         # combine masks for red  
۴۵         mask = np.zeros(hsv.shape[:2], dtype=np.uint8)  
۴۶         for lo, hi in ranges:  
۴۷             lo = np.array(lo, dtype=np.uint8)  
۴۸             hi = np.array(hi, dtype=np.uint8)  
۴۹             mask |= cv2.inRange(hsv, lo, hi)  
۵۰  
۵۱         # morphological separation  
۵۲         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, open_kernel)  
۵۳         mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, close_kernel)  
۵۴         mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, separate_kernel)  
۵۵  
۵۶     # connected components  
۵۷     n_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(  
۵۸         mask, connectivity=8
```



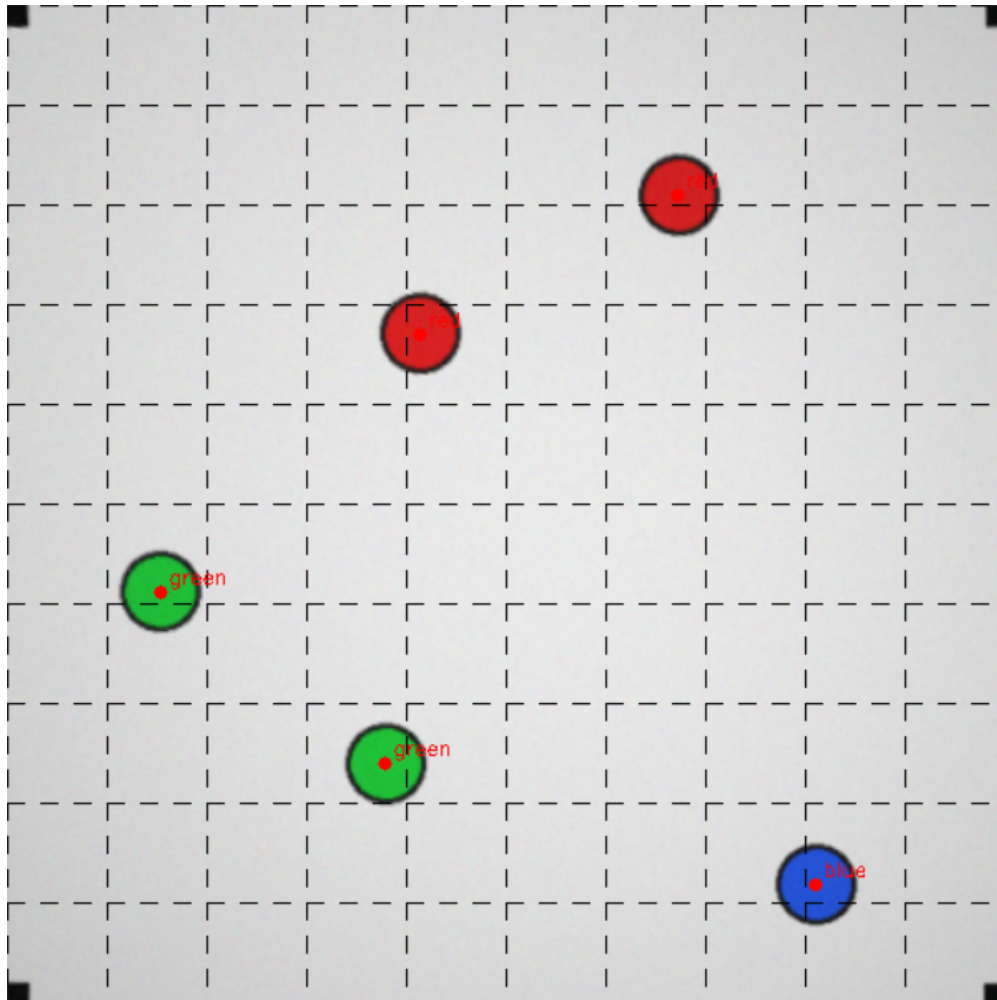
```
۵۹     )
۶۰
۶۱     # random color for components (displayed only)
۶۲     # shows separate images for separate disks on black background
۶۳     comp_vis = np.zeros((mask.shape[0], mask.shape[1], 3), dtype=np.uint8)
۶۴     rng = np.random.default_rng(42)
۶۵
۶۶     for label in range(1, n_labels): # skip background
۶۷         area = stats[label, cv2.CC_STAT_AREA]
۶۸         if area < 400: # for small noise
۶۹             continue
۷۰
۷۱         # centroid
۷۲         cx, cy = centroids[label]
۷۳         u = int(round(cx))
۷۴         v = int(round(cy))
۷۵
۷۶         # store results
۷۷         results.append((color_name, u, v))
۷۸
۷۹         # draw centroid
۸۰         cv2.circle(vis_img, (u, v), 4, (0, 0, 255), -1)
۸۱         cv2.putText(
۸۲             vis_img,
۸۳             f"{color_name}",
۸۴             (u + 5, v - 5),
۸۵             cv2.FONT_HERSHEY_SIMPLEX,
۸۶             0.4,
۸۷             (0, 0, 255),
۸۸             1
۸۹         )
۹۰
۹۱         color = rng.integers(50, 255, size=3, dtype=np.uint8)
۹۲         comp_vis[labels == label] = color
۹۳
۹۴     # show per color imgs
۹۵     cv2.imshow(f"{color_name} components (separated)", comp_vis)
۹۶
۹۷     # final
۹۸     cv2.imshow("Original with centroids", vis_img)
```



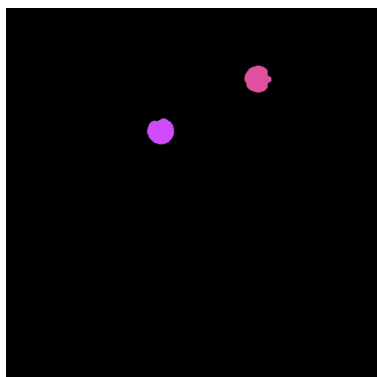
```
۹۹
۱۰۰     # tuple out
۱۰۱     tuple_output = [(color, (u, v)) for (color, u, v) in results]
۱۰۲     print("Detected discs (color, (u, v)):")
۱۰۳     for t in tuple_output:
۱۰۴         print(t)
۱۰۵
۱۰۶     cv2.waitKey(0)
۱۰۷     cv2.destroyAllWindows()
۱۰۸
۱۰۹     return tuple_output
۱۱۰
۱۱۱
۱۱۲ # run
۱۱۳ path_to_png = r"C:\Users\Asus\Desktop\hasti\3.png" # edit this as needed
۱۱۴ results = segment_discs_and_show(path_to_png)
۱۱۵
```

خروجی کد

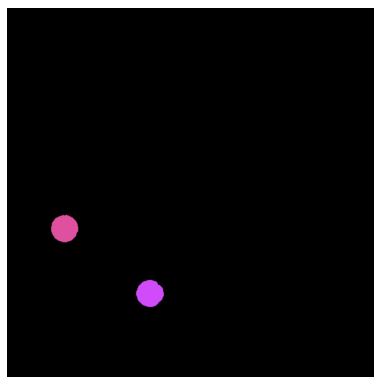
در کد این بخش، علاوه بر مشخص کردن دیسک‌ها بر روی خروجی مرحله قبلی (شکل ۴) که در شکل ۵ قابل مشاهده است، دیسک‌ها را به صورت تفکیک‌شده بر اساس رنگ، در تصویرهای مستقل نمایش داده‌ایم که در شکل ۶ قابل مشاهده هستند.



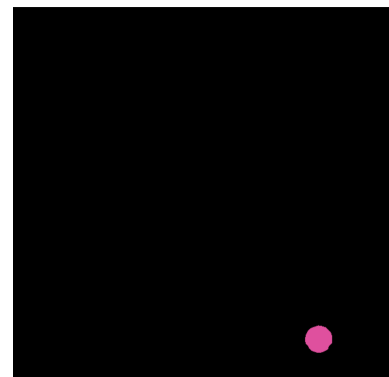
شکل ۵: مشخص کردن نشانگرها روی تصویر به ترتیب ساعتگرد



(ج) دیسک‌های قرمز



(ب) دیسک‌های سبز



(آ) دیسک آبی

شکل ۶: تصویرهای مستقل از دیسک‌ها با تفکیک بر اساس رنگ

علاوه بر این تصاویر، مختصات مرکز ثقل دیسک‌ها نیز به صورت Tupple خروجی دیگر این کد است:

Detected corners:

```
[[127. 106.]  
 [498.  62.]  
 [520. 439.]  
 [105. 455.]]
```

Detected discs (color, (u, v)):

```
('red', (403, 114))  
('red', (248, 198))  
('green', (92, 353))  
('green', (227, 456))  
('blue', (486, 529))
```

۵.۱.۲ تسک A.5 — تبدیل مختصات به دستگاه ربات و تکمیل نهایی پایپ لاین

در گام آخر، مختصات پیکسلی مرکز هر دیسک را با استفاده تناسب تعریف شده در مرحله قبل به مختصات واقعی و متریک تبدیل می‌کنیم. سپس تمام توابع نوشته شده در تسک‌های پیشین را در یک تابع به نام `vision_pipeline` تجمیع کرده و مختصات محاسبه شده را با مختصات واقعی قطعات (Ground-truth) مقایسه کرده و خطای الگوریتم بینایی را به دست می‌آوریم.

به دست آوردن مختصات واقعی و خطای الگوریتم بینایی با توجه به این که هر یک متر در دنیای واقعی برابر با ۱۰۰۰ پیکسل در نظر گرفته شده است، با استفاده از تابع زیر، مختصات واقعی مرکز ثقل دیسک‌ها را به دست می‌آوریم:

```
1 def world_to_pixel(world_x, world_y, center_u=300, center_v=300, scale=1000):  
2     u = int(center_u - world_y * scale)  
3     v = int(center_v - world_x * scale)  
4     return (u, v)
```

خروجی این تابع

```
red: pixel (404, 114) -> world (+0.186, -0.104)  
red: pixel (248, 197) -> world (+0.103, +0.052)  
green: pixel (92, 353) -> world (-0.053, +0.208)  
green: pixel (228, 456) -> world (-0.156, +0.072)  
blue: pixel (486, 529) -> world (-0.229, -0.186)
```

است.

سپس با تابع دیگری که در زیر تعریف شده، این مختصات را با مختصات Ground Truth مقایسه می‌کنیم و خطای الگوریتم بینایی را به دست می‌آوریم:



```
1 def match_discs_to_ground_truth(detected_discs, ground_truth):
2     detected_world = [
3         {"color": color, "world_xy": pixel_to_world(u, v), "pixel_uv": (u, v)}
4         for color, u, v in detected_discs
5     ]
6
7     gt_remaining = ground_truth.copy()
8     matches = []
9
10    for det in detected_world:
11        best_match = None
12        best_distance = float('inf')
13
14        for gt in gt_remaining:
15            if gt["color"] != det["color"]:
16                continue
17
18            dx = det["world_xy"][0] - gt["world_xy"][0]
19            dy = det["world_xy"][1] - gt["world_xy"][1]
20            distance = np.sqrt(dx**2 + dy**2)
21
22            if distance < best_distance:
23                best_distance = distance
24                best_match = gt
25
26        if best_match:
27            error_x = det["world_xy"][0] - best_match["world_xy"][0]
28            error_y = det["world_xy"][1] - best_match["world_xy"][1]
29
30            matches.append({
31                "color": det["color"],
32                "detected_world_xy": det["world_xy"],
33                "detected_pixel_uv": det["pixel_uv"],
34                "ground_truth_world_xy": best_match["world_xy"],
35                "error_xy": (error_x, error_y),
36                "error_magnitude": best_distance
37            })
38
39        gt_remaining.remove(best_match)
40
41    return matches, gt_remaining
```

خروجی این تابع به صورت جدول با دقت میلی متر در پایین قابل مشاهده است.

Color	Detected (world)	Ground Truth	Error	Magnitude
red	(+0.186, -0.104)	(+0.180, -0.100)	(+0.006, -0.004)	0.0072
red	(+0.103, +0.052)	(+0.100, +0.050)	(+0.003, +0.002)	0.0036
green	(-0.053, +0.208)	(-0.050, +0.200)	(-0.003, +0.008)	0.0085
green	(-0.156, +0.072)	(-0.150, +0.070)	(-0.006, +0.002)	0.0063
blue	(-0.229, -0.186)	(-0.220, -0.180)	(-0.009, -0.006)	0.0108

تجمع توابع

با تجمع همه توابع نوشته شده در تسک‌های پیشین و اضافه کردن دو تابع دیگر برای بدست آوردن مختصات واقعی و همچنین پیدا کردن خطا، در نهایت پایپ‌لاین کامل به صورت زیر پیاده‌سازی می‌شود:

```

1 import cv2
2 import numpy as np
3 from pathlib import Path
4
5
6 GROUND_TRUTH_DISCS = [
7     {"color": "red", "world_xy": (+0.18, -0.10)},
8     {"color": "green", "world_xy": (-0.05, +0.20)},
9     {"color": "blue", "world_xy": (-0.22, -0.18)},
10    {"color": "red", "world_xy": (+0.10, +0.05)},
11    {"color": "green", "world_xy": (-0.15, +0.07)},
12 ]
13
14
15 def detect_corner_markers(image):
16     hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
17
18     _, binary = cv2.threshold(
19         hsv[:, :, 2],
20         50,
21         255,
22         cv2.THRESH_BINARY_INV
23     )
24 
```



```
۲۵     contours, _ = cv2.findContours(  
۲۶         binary,  
۲۷         cv2.RETR_EXTERNAL,  
۲۸         cv2.CHAIN_APPROX_SIMPLE  
۲۹     )  
۳۰  
۳۱     square_candidates = []  
۳۲  
۳۳     for cnt in contours:  
۳۴         area = cv2.contourArea(cnt)  
۳۵  
۳۶         if area < 100:  
۳۷             continue  
۳۸  
۳۹         x, y, w, h = cv2.boundingRect(cnt)  
۴۰         box_area = w * h  
۴۱         squareness = area / box_area if box_area > 0 else 0  
۴۲  
۴۳         if squareness > 0.3:  
۴۴             square_candidates.append(cnt)  
۴۵  
۴۶     square_candidates.sort(key=cv2.contourArea, reverse=True)  
۴۷     top4 = square_candidates[:4]  
۴۸  
۴۹     centers = []  
۵۰     for cnt in top4:  
۵۱         M = cv2.moments(cnt)  
۵۲  
۵۳         if M['m00'] == 0:  
۵۴             continue  
۵۵  
۵۶         cx = int(M['m10'] / M['m00'])  
۵۷         cy = int(M['m01'] / M['m00'])  
۵۸         centers.append((cx, cy))  
۵۹  
۶۰     centers = sort_clockwise(centers)  
۶۱     return np.float32(centers)  
۶۲  
۶۳  
۶۴ def sort_clockwise(pts):
```



```
۶۵ pts = sorted(pts, key=lambda p: p[1])
۶۶ top = sorted(pts[:2], key=lambda p: p[0])
۶۷ bottom = sorted(pts[2:], key=lambda p: p[0], reverse=True)
۶۸ return [top[0], top[1], bottom[0], bottom[1]]
۶۹
۷۰
۷۱ def compute_center(corners):
۷۲     p1, p2, p3, p4 = corners[0], corners[2], corners[1], corners[3]
۷۳
۷۴     x1, y1 = p1.astype(np.float64)
۷۵     x2, y2 = p2.astype(np.float64)
۷۶     x3, y3 = p3.astype(np.float64)
۷۷     x4, y4 = p4.astype(np.float64)
۷۸
۷۹     A1 = y2 - y1
۸۰     B1 = x1 - x2
۸۱     C1 = A1 * x1 + B1 * y1
۸۲
۸۳     A2 = y4 - y3
۸۴     B2 = x3 - x4
۸۵     C2 = A2 * x3 + B2 * y3
۸۶
۸۷     det = A1 * B2 - A2 * B1
۸۸
۸۹     center_x = (B2 * C1 - B1 * C2) / det
۹۰     center_y = (A1 * C2 - A2 * C1) / det
۹۱
۹۲     return np.array([[center_x, center_y]], dtype=np.float32)
۹۳
۹۴
۹۵ def rectify_image(image, corners, canvas_size=600):
۹۶     dst_points = np.float32([
۹۷         [0, 0],
۹۸         [canvas_size - 1, 0],
۹۹         [canvas_size - 1, canvas_size - 1],
۱۰۰         [0, canvas_size - 1]
۱۰۱     ])
۱۰۲
۱۰۳     H, _ = cv2.findHomography(corners, dst_points)
۱۰۴
```



```
105 rectified = cv2.warpPerspective(  
106     image,  
107     H,  
108     (canvas_size, canvas_size)  
109 )  
110  
111 return rectified, H  
112  
113  
114 def draw_grid(image, canvas_size=600, grid_spacing=60, dash_length=10, gap_length=10):  
115     result = image.copy()  
116  
117     for x in range(0, canvas_size, grid_spacing):  
118         y = 0  
119         while y < canvas_size:  
120             y_end = min(y + dash_length, canvas_size)  
121             cv2.line(result, (x, y), (x, y_end), (0, 0, 0), 1)  
122             y += dash_length + gap_length  
123  
124     for y in range(0, canvas_size, grid_spacing):  
125         x = 0  
126         while x < canvas_size:  
127             x_end = min(x + dash_length, canvas_size)  
128             cv2.line(result, (x, y), (x_end, y), (0, 0, 0), 1)  
129             x += dash_length + gap_length  
130  
131     return result  
132  
133  
134 def segment_discs(rectified_img):  
135     hsv = cv2.cvtColor(rectified_img, cv2.COLOR_BGR2HSV)  
136     results = []  
137  
138     color_ranges = {  
139         'red': [  
140             ([0, 100, 100], [10, 255, 255]),  
141             ([170, 100, 100], [180, 255, 255])  
142         ],  
143         'green': [  
144             ([40, 80, 80], [80, 255, 255])
```



```
145     ],
146     'blue': [
147         ([100, 80, 80], [130, 255, 255])
148     ],
149 }
150
151 open_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))
152 close_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (9, 9))
153 separate_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))
154
155 for color_name, ranges in color_ranges.items():
156     mask = np.zeros(hsv.shape[:2], dtype=np.uint8)
157
158     for lo, hi in ranges:
159         lo = np.array(lo, dtype=np.uint8)
160         hi = np.array(hi, dtype=np.uint8)
161         mask |= cv2.inRange(hsv, lo, hi)
162
163     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, open_kernel)
164     mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, close_kernel)
165     mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, separate_kernel)
166
167     n_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
168         mask, connectivity=8
169     )
170
171     for label in range(1, n_labels):
172         area = stats[label, cv2.CC_STAT_AREA]
173         if area < 400:
174             continue
175
176         cx, cy = centroids[label]
177         u = int(round(cx))
178         v = int(round(cy))
179
180         results.append((color_name, u, v))
181
182     return results
183
184
```




```
185 def pixel_to_world(u, v, center_u=300, center_v=300, scale=1000):
186     world_x = (center_v - v) / scale
187     world_y = (center_u - u) / scale
188     return (world_x, world_y)
189
190
191 def world_to_pixel(world_x, world_y, center_u=300, center_v=300, scale=1000):
192     u = int(center_u - world_y * scale)
193     v = int(center_v - world_x * scale)
194     return (u, v)
195
196
197 def match_discs_to_ground_truth(detected_discs, ground_truth):
198     detected_world = [
199         {"color": color, "world_xy": pixel_to_world(u, v), "pixel_uv": (u, v)}
200         for color, u, v in detected_discs
201     ]
202
203     gt_remaining = ground_truth.copy()
204     matches = []
205
206     for det in detected_world:
207         best_match = None
208         best_distance = float('inf')
209
210         for gt in gt_remaining:
211             if gt["color"] != det["color"]:
212                 continue
213
214             dx = det["world_xy"][0] - gt["world_xy"][0]
215             dy = det["world_xy"][1] - gt["world_xy"][1]
216             distance = np.sqrt(dx**2 + dy**2)
217
218             if distance < best_distance:
219                 best_distance = distance
220                 best_match = gt
221
222     if best_match:
223         error_x = det["world_xy"][0] - best_match["world_xy"][0]
224         error_y = det["world_xy"][1] - best_match["world_xy"][1]
```



```
۲۲۵
۲۲۶     matches.append({
۲۲۷         "color": det["color"],
۲۲۸         "detected_world_xy": det["world_xy"],
۲۲۹         "detected_pixel_uv": det["pixel_uv"],
۲۳۰         "ground_truth_world_xy": best_match["world_xy"],
۲۳۱         "error_xy": (error_x, error_y),
۲۳۲         "error_magnitude": best_distance
۲۳۳     })
۲۳۴
۲۳۵     gt_remaining.remove(best_match)
۲۳۶
۲۳۷     return matches, gt_remaining
۲۳۸
۲۳۹
۲۴۰ def vision_pipeline(image_path, output_dir="./output", canvas_size=600,
۲۴۱                       grid_spacing=60, visualize=True):
۲۴۲     output_dir = Path(output_dir)
۲۴۳     output_dir.mkdir(exist_ok=True)
۲۴۴
۲۴۵     image = cv2.imread(str(image_path))
۲۴۶
۲۴۷     print("="*60)
۲۴۸     print("VISION PIPELINE")
۲۴۹     print("="*60)
۲۵۰
۲۵۱     print("\nStep 1: Detecting corner markers...")
۲۵۲     corners = detect_corner_markers(image)
۲۵۳     print(f"Detected corners:\n{corners}")
۲۵۴
۲۵۵     print("\nStep 2: Computing center and rectifying...")
۲۵۶     center_original = compute_center(corners)
۲۵۷     rectified_image, homography = rectify_image(image, corners, canvas_size)
۲۵۸     center_rectified = cv2.perspectiveTransform(center_original, homography)
۲۵۹
۲۶۰     cx_orig = int(round(center_original[0][0][0]))
۲۶۱     cy_orig = int(round(center_original[0][0][1]))
۲۶۲     cx_rect = int(round(center_rectified[0][0][0]))
۲۶۳     cy_rect = int(round(center_rectified[0][0][1]))
۲۶۴
```



```

۲۶۵     print(f"Original center: ({cx_orig}, {cy_orig})")
۲۶۶     print(f"Rectified center: ({cx_rect}, {cy_rect})")
۲۶۷
۲۶۸     print("\nCorner world coordinates:")
۲۶۹     corner_labels = ["Top-Left", "Top-Right", "Bottom-Right", "Bottom-Left"]
۲۷۰     expected_coords = [(0.3, 0.3), (0.3, -0.3), (-0.3, -0.3), (-0.3, 0.3)]
۲۷۱     for i, (label, expected) in enumerate(zip(corner_labels, expected_coords)):
۲۷۲         u, v = 0 if i % 2 == 0 else canvas_size - 1, 0 if i < 2 else canvas_size - 1
۲۷۳         world_x, world_y = pixel_to_world(u, v)
۲۷۴         print(f"  {label}: pixel ({u}, {v}) -> world ({world_x:+.3f}, {world_y:+.3f}) [expected:
           ↪ {expected}]")
۲۷۵
۲۷۶     rectified_with_grid = draw_grid(rectified_image, canvas_size, grid_spacing)
۲۷۷
۲۷۸     print("\nStep 3: Segmenting colored discs...")
۲۷۹     detected_discs = segment_discs(rectified_image)
۲۸۰
۲۸۱     print(f"\nDetected {len(detected_discs)} discs (pixel coordinates):")
۲۸۲     for color, u, v in detected_discs:
۲۸۳         print(f"  {color}: ({u}, {v})")
۲۸۴
۲۸۵     print("\nStep 4: Converting to world coordinates...")
۲۸۶     discs_world = []
۲۸۷     for color, u, v in detected_discs:
۲۸۸         world_x, world_y = pixel_to_world(u, v)
۲۸۹         discs_world.append({
۲۹۰             "color": color,
۲۹۱             "pixel_uv": (u, v),
۲۹۲             "world_xy": (world_x, world_y)
۲۹۳         })
۲۹۴         print(f"  {color}: pixel ({u}, {v}) -> world ({world_x:+.3f}, {world_y:+.3f})")
۲۹۵
۲۹۶     print("\nStep 5: Comparing with ground truth...")
۲۹۷     matches, unmatched_gt = match_discs_to_ground_truth(detected_discs, GROUND_TRUTH_DISCS)
۲۹۸
۲۹۹     print(f"\nMatched {len(matches)} discs:")
۳۰۰     print("\n" + "-"*80)
۳۰۱     print(f"{'Color':<8} {'Detected (world)':<20} {'Ground Truth':<20} {'Error':<20}
           ↪ {'Magnitude':<10}")
۳۰۲     print("-"*80)

```



```

۳۰۳
۳۰۴     for match in matches:
۳۰۵         det_str = f"({match['detected_world_xy'][0]:+.3f}, {match['detected_world_xy'][1]:+.3f})"
۳۰۶         gt_str = f"({match['ground_truth_world_xy'][0]:+.3f},
        ↳ {match['ground_truth_world_xy'][1]:+.3f})"
۳۰۷         err_str = f"({match['error_xy'][0]:+.3f}, {match['error_xy'][1]:+.3f})"
۳۰۸         mag_str = f"{match['error_magnitude']:.4f}"
۳۰۹
۳۱۰         print(f"{match['color']:<8} {det_str:<20} {gt_str:<20} {err_str:<20} {mag_str:<10}")
۳۱۱
۳۱۲     if unmatched_gt:
۳۱۳         print(f"\nUnmatched ground truth discs: {len(unmatched_gt)}")
۳۱۴         for gt in unmatched_gt:
۳۱۵             print(f"    {gt['color']}: {gt['world_xy']}")
۳۱۶
۳۱۷     if visualize:
۳۱۸         vis_corners = image.copy()
۳۱۹         labels = ["TL", "TR", "BR", "BL"]
۳۲۰         for i, (x, y) in enumerate(corners.astype(int)):
۳۲۱             cv2.circle(vis_corners, (x, y), 10, (0, 0, 255), -1)
۳۲۲             cv2.putText(vis_corners, labels[i], (x + 10, y - 10),
۳۲۳                         cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
۳۲۴
۳۲۵         vis_center = image.copy()
۳۲۶         cv2.circle(vis_center, (cx_orig, cy_orig), 4, (255, 0, 0), -1)
۳۲۷         cv2.putText(vis_center, "CENTER", (cx_orig + 8, cy_orig),
۳۲۸                     cv2.FONT_HERSHEY_SIMPLEX, 0.2, (255, 0, 0), 1)
۳۲۹
۳۳۰         vis_rectified = rectified_with_grid.copy()
۳۳۱         cv2.circle(vis_rectified, (cx_rect, cy_rect), 4, (0, 0, 255), -1)
۳۳۲         cv2.putText(vis_rectified, "CENTER", (cx_rect + 8, cy_rect),
۳۳۳                     cv2.FONT_HERSHEY_SIMPLEX, 0.2, (0, 0, 255), 1)
۳۳۴
۳۳۵         vis_discs = rectified_image.copy()
۳۳۶         for color, u, v in detected_discs:
۳۳۷             cv2.circle(vis_discs, (u, v), 4, (0, 0, 255), -1)
۳۳۸             cv2.putText(vis_discs, color, (u + 5, v - 5),
۳۳۹                         cv2.FONT_HERSHEY_SIMPLEX, 0.4, (0, 0, 255), 1)
۳۴۰
۳۴۱         vis_comparison = rectified_image.copy()

```



```
۳۴۲
۳۴۳     cv2.circle(vis_comparison, (300, 300), 5, (128, 128, 128), -1)
۳۴۴
۳۴۵     for gt in GROUND_TRUTH_DISCS:
۳۴۶         gt_u, gt_v = world_to_pixel(gt["world_xy"][0], gt["world_xy"][1])
۳۴۷         cv2.circle(vis_comparison, (gt_u, gt_v), 8, (0, 255, 0), 2)
۳۴۸         cv2.putText(vis_comparison, f"GT-{gt['color']}", (gt_u + 10, gt_v - 10),
۳۴۹                     cv2.FONT_HERSHEY_SIMPLEX, 0.3, (0, 255, 0), 1)
۳۵۰
۳۵۱     for match in matches:
۳۵۲         det_u, det_v = match['detected_pixel_uv']
۳۵۳         gt_u, gt_v = world_to_pixel(match['ground_truth_world_xy'][0],
۳۵۴                                     match['ground_truth_world_xy'][1])
۳۵۵
۳۵۶         cv2.circle(vis_comparison, (det_u, det_v), 6, (0, 0, 255), -1)
۳۵۷         cv2.line(vis_comparison, (det_u, det_v), (gt_u, gt_v), (255, 0, 255), 1)
۳۵۸
۳۵۹         mid_u = (det_u + gt_u) // 2
۳۶۰         mid_v = (det_v + gt_v) // 2
۳۶۱         cv2.putText(vis_comparison, f"{match['error_magnitude']:.3f}",
۳۶۲                     (mid_u, mid_v), cv2.FONT_HERSHEY_SIMPLEX, 0.3, (255, 0, 255), 1)
۳۶۳
۳۶۴     cv2.imwrite(str(output_dir / "1_detected_corners.jpg"), vis_corners)
۳۶۵     cv2.imwrite(str(output_dir / "2_original_center.jpg"), vis_center)
۳۶۶     cv2.imwrite(str(output_dir / "3_rectified_grid.jpg"), vis_rectified)
۳۶۷     cv2.imwrite(str(output_dir / "4_segmented_discs.jpg"), vis_discs)
۳۶۸     cv2.imwrite(str(output_dir / "5_error_comparison.jpg"), vis_comparison)
۳۶۹     cv2.imwrite(str(output_dir / "rectified.png"), rectified_image)
۳۷۰
۳۷۱     print(f"\nVisualizations saved to {output_dir}/")
۳۷۲
۳۷۳     errors_x = [m['error_xy'][0] for m in matches]
۳۷۴     errors_y = [m['error_xy'][1] for m in matches]
۳۷۵     errors_mag = [m['error_magnitude'] for m in matches]
۳۷۶
۳۷۷     return {
۳۷۸         'corners': corners,
۳۷۹         'center_original': (cx_orig, cy_orig),
۳۸۰         'center_rectified': (cx_rect, cy_rect),
۳۸۱         'homography': homography,
```



```

۳۸۲     'rectified_image': rectified_image,
۳۸۳     'detected_discs_pixel': [(color, (u, v)) for color, u, v in detected_discs],
۳۸۴     'detected_discs_world': discs_world,
۳۸۵     'matches': matches,
۳۸۶     'unmatched_ground_truth': unmatched_gt,
۳۸۷     'error_statistics': {
۳۸۸         'mean_error_x': np.mean(errors_x) if matches else None,
۳۸۹         'mean_error_y': np.mean(errors_y) if matches else None,
۳۹۰         'rms_error_x': np.sqrt(np.mean(np.array(errors_x)**2)) if matches else None,
۳۹۱         'rms_error_y': np.sqrt(np.mean(np.array(errors_y)**2)) if matches else None,
۳۹۲         'mean_magnitude': np.mean(errors_mag) if matches else None,
۳۹۳         'max_magnitude': np.max(errors_mag) if matches else None,
۳۹۴     }
۳۹۵ }
۳۹۶
۳۹۷
۳۹۸ if __name__ == "__main__":
۳۹۹     image_path = r"C:\Users\Asus\Desktop\hasti\output.png"
۴۰۰
۴۰۱     results = vision_pipeline(
۴۰۲         image_path=image_path,
۴۰۳         output_dir="./output",
۴۰۴         canvas_size=600,
۴۰۵         grid_spacing=60,
۴۰۶         visualize=True
۴۰۷     )
۴۰۸
۴۰۹     print("\n" + "="*60)
۴۱۰     print("PIPELINE COMPLETE")
۴۱۱     print("="*60)
۴۱۲

```

که خروجی آن به صورت زیر است:

```

=====
VISION PIPELINE
=====

```

Step 1: Detecting corner markers...

Detected corners:



```
[[ 49.  72.]  
 [420.  28.]  
 [442. 405.]  
 [ 27. 421.]]
```

Step 2: Computing center and rectifying...

Original center: (226, 222)

Rectified center: (300, 300)

Corner world coordinates:

```
Top-Left: pixel (0, 0) -> world (+0.300, +0.300) [expected: (0.3, 0.3)]  
Top-Right: pixel (599, 0) -> world (+0.300, -0.299) [expected: (0.3, -0.3)]  
Bottom-Right: pixel (0, 599) -> world (-0.299, +0.300) [expected: (-0.3, -0.3)]  
Bottom-Left: pixel (599, 599) -> world (-0.299, -0.299) [expected: (-0.3, 0.3)]
```

Step 3: Segmenting colored discs...

Detected 5 discs (pixel coordinates):

```
red: (404, 114)  
red: (248, 197)  
green: (92, 353)  
green: (228, 456)  
blue: (486, 529)
```

Step 4: Converting to world coordinates...

```
red: pixel (404, 114) -> world (+0.186, -0.104)  
red: pixel (248, 197) -> world (+0.103, +0.052)  
green: pixel (92, 353) -> world (-0.053, +0.208)  
green: pixel (228, 456) -> world (-0.156, +0.072)  
blue: pixel (486, 529) -> world (-0.229, -0.186)
```

Step 5: Comparing with ground truth...

Matched 5 discs:



Color	Detected (world)	Ground Truth	Error	Magnitude
red	(+0.186, -0.104)	(+0.180, -0.100)	(+0.006, -0.004)	0.0072
red	(+0.103, +0.052)	(+0.100, +0.050)	(+0.003, +0.002)	0.0036
green	(-0.053, +0.208)	(-0.050, +0.200)	(-0.003, +0.008)	0.0085
green	(-0.156, +0.072)	(-0.150, +0.070)	(-0.006, +0.002)	0.0063
blue	(-0.229, -0.186)	(-0.220, -0.180)	(-0.009, -0.006)	0.0108

Visualizations saved to output/

PIPELINE COMPLETE

۲.۲ بخش B — سینماتیک مستقیم (Forward Kinematics) ربات PRR

معرفی ربات و ساختار کلی آن

ربات مورد بررسی یک ربات بازرس از نوع PRR است که از سه مفصل تشکیل شده است. حرف P نشان‌دهنده یک مفصل منشوری (کشویی) و دو حرف R نشان‌دهنده دو مفصل لولایی (دورانی) می‌باشند. ساختار این ربات به گونه‌ای است که مفصل اول وظیفه تنظیم ارتفاع کل بازو را بر عهده دارد و دو مفصل بعدی، مانند یک بازوی صفحه‌ای (Planar)، در یک صفحه افقی حرکت می‌کنند تا به نقطه هدف برسند.

مشخصات مفاصل و پارامترهای فیزیکی

بر اساس اطلاعات موجود، پارامترهای حرکتی و ابعاد فیزیکی ربات به شرح زیر است:

- مفصل ۱ (کشویی): این مفصل در امتداد محور Z جهانی حرکت می‌کند. دامنه حرکتی آن $q_1 \in [0, 0.50]$ متر است. ارتفاع اولیه محور مفصل دوم از سطح زمین برابر با $D_0 = 0.70$ متر می‌باشد.
- مفصل ۲ (دورانی): این مفصل حول محور Z دوران می‌کند و قابلیت چرخش کامل به صورت $q_2 \in [-\pi, +\pi]$ را دارد. طول لینک آن $L_2 = 0.40$ متر است.
- مفصل ۳ (دورانی): این مفصل نیز حول محور Z می‌چرخد و دامنه حرکتی آن $q_3 \in [-\pi, +\pi]$ است. طول لینک آن $L_3 = 0.30$ متر می‌باشد.

نقطه ابزار (افکتور نهایی) در محل تقاطع محور دورانی سوم با نوک لینک آخر قرار دارد و طول ابزار متصل به آن صفر در نظر گرفته شده است.



۱.۲.۲ تسک B.1 - جدول پارامترهای دناویت-هاروتنبرگ (DH)

با استفاده از روش استاندارد DH، جدول پارامترهای این ربات به شکل زیر تدوین می‌شود (متغیرهای مفصلی به صورت پررنگ مشخص شده‌اند):

لینک i	α_{i-1} (رادیان)	a_{i-1} (متر)	d_i (متر)	θ_i (رادیان)	نوع مفصل
۱	۰	۰	$D_0 + q_1$	۰	کشویی (P)
۲	۰	L_2	۰	q_2	دورانی (R)
۳	۰	L_3	۰	q_3	دورانی (R)

هر سطر از این جدول برای ساخت ماتریس تبدیل مربوط به همان لینک استفاده می‌شود. فرمول کلی ساخت ماتریس انتقال در این روش به صورت ضرب چهار ماتریس پایه زیر است:

$${}^{i-1}T_i = R_z(\theta_i)T_x(d_i)T_x(a_{i-1})R_x(\alpha_{i-1})$$

نحوه کارکرد و تحلیل حرکتی

از نظر سینماتیکی، موقعیت نهایی ربات ترکیبی از این سه حرکت است. مفصل اول با تغییر متغیر q_1 ، مختصات ارتفاع (محور Z) نقطه نهایی را مستقیماً تعیین می‌کند. از آنجایی که α در تمامی مفاصل صفر است، صفحات حرکتی مفاصل دوم و سوم موازی با صفحه افقی (XY) باقی می‌مانند. مفاصل دوم و سوم به ترتیب با زوایای q_2 و q_3 و با استفاده از طول بازوهای L_2 و L_3 ، موقعیت افکتور را در این صفحه افقی مشخص می‌کنند. با ضرب ماتریس‌های تبدیل حاصل از جدول بالا، می‌توان معادلات دقیق موقعیت x ، y و z افکتور نهایی را بر حسب مقادیر مفصلی محاسبه نمود.

۲.۲.۲ تسک B.2 — محاسبات نمادین با SymPy

نحوه پیاده‌سازی کد و محاسبه ماتریس‌ها

برای محاسبه سینماتیک مستقیم این ربات به صورت پارامتری، از کتابخانه قدرتمند SymPy در زبان پایتون استفاده شده است. در ابتدا، متغیرهای مفصلی (q_1, q_2, q_3) و پارامترهای ثابت فیزیکی (D_0, L_2, L_3) به عنوان متغیرهای نمادین (Symbolic) تعریف شده‌اند. سپس تابعی به نام `dh_matrix` نوشته شده است که با دریافت چهار پارامتر استاندارد دناویت-هاروتنبرگ (θ, d, a, α) ، ماتریس تبدیل همگن 4×4 مربوطه را تولید می‌کند. بر اساس جدول DH، ماتریس تبدیل هر لینک به صورت مجزا (T_1^0, T_2^1, T_3^2) ساخته شده و در نهایت با ضرب متوالی آن‌ها $(T_1^0 \times T_2^1 \times T_3^2)$ ، ماتریس تبدیل نهایی T_3^0 به دست آمده است. در انتها با استفاده از دستور `sp.simplify`، عبارات مثلثاتی ساده‌سازی شده و ستون چهارم ماتریس به عنوان مختصات استخراج گردیده است.

کد پایتون پیاده‌سازی شده

کد استفاده شده برای این محاسبات به شرح زیر است:

```
import sympy as sp
```

```
۲
۳ # Initialize printing for pretty formatting in Jupyter Notebook
۴ sp.init_printing(use_latex='mathjax')
۵
۶ # 1. Define symbolic variables
۷ q1, q2, q3 = sp.symbols('q1 q2 q3')
۸ D0, L2, L3 = sp.symbols('D0 L2 L3')
۹
۱۰ # Define DH transformation matrix function
۱۱ def dh_matrix(theta, d, a, alpha):
۱۲     return sp.Matrix([
۱۳         [sp.cos(theta), -sp.sin(theta)*sp.cos(alpha), sp.sin(theta)*sp.sin(alpha), a*sp.cos(theta)],
۱۴         [sp.sin(theta), sp.cos(theta)*sp.cos(alpha), -sp.cos(theta)*sp.sin(alpha), a*sp.sin(theta)],
۱۵         [0, sp.sin(alpha), sp.cos(alpha), d],
۱۶         [0, 0, 0, 1]
۱۷     ])
۱۸
۱۹ # 2. Construct transformation matrices for each link based on the DH table
۲۰ T1_0 = dh_matrix(0, D0 + q1, 0, 0)
۲۱ T2_1 = dh_matrix(q2, 0, L2, 0)
۲۲ T3_2 = dh_matrix(q3, 0, L3, 0)
۲۳
۲۴ # 3. Calculate final transformation matrix (OT3 = T1_0 * T2_1 * T3_2)
۲۵ T3_0 = T1_0 * T2_1 * T3_2
۲۶ T3_0_simplified = sp.simplify(T3_0)
۲۷
۲۸ # 4. Extract position coordinates (x, y, z)
۲۹ x = T3_0_simplified[0, 3]
۳۰ y = T3_0_simplified[1, 3]
۳۱ z = T3_0_simplified[2, 3]
۳۲
۳۳ # 5. Display results clearly
۳۴ print("--- Final Transformation Matrix (OT3) ---")
۳۵ display(T3_0_simplified)
۳۶
۳۷ print("\n--- End-Effector Position Equations (Symbolic) ---")
۳۸ display(sp.Matrix([x, y, z]))
۳۹
۴۰ # 6. Numerical substitution for specific link lengths
۴۱ constants = {D0: 0.70, L2: 0.40, L3: 0.30}
```



```

۴۲ x_num = x.subs(constants)
۴۳ y_num = y.subs(constants)
۴۴ z_num = z.subs(constants)
۴۵
۴۶ print("\n--- End-Effector Position Equations (with given constants) ---")
۴۷ print(f"x = {x_num}")
۴۸ print(f"y = {y_num}")
۴۹ print(f"z = {z_num}")

```

خروجی های به دست آمده (ماتریس و معادلات موقعیت)

با اجرای کد بالا، ماتریس تبدیل نهایی 0T_3 به شکل ساده شده زیر به دست می آید:

$${}^0T_3 = \begin{bmatrix} \cos(q_2 + q_3) & -\sin(q_2 + q_3) & 0 & L_2 \cos(q_2) + L_3 \cos(q_2 + q_3) \\ \sin(q_2 + q_3) & \cos(q_2 + q_3) & 0 & L_2 \sin(q_2) + L_3 \sin(q_2 + q_3) \\ 0 & 0 & 1 & D_0 + q_1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

معادلات موقعیت مجری نهایی (End-Effector) به صورت نمادین:

$$x = L_2 \cos(q_2) + L_3 \cos(q_2 + q_3)$$

$$y = L_2 \sin(q_2) + L_3 \sin(q_2 + q_3)$$

$$z = D_0 + q_1$$

با جایگذاری مقادیر عددی ($L_3 = 0.30$, $L_2 = 0.40$, $D_0 = 0.70$)، معادلات به فرم زیر در می آیند:

$$x = 0.4 \cos(q_2) + 0.3 \cos(q_2 + q_3)$$

$$y = 0.4 \sin(q_2) + 0.3 \sin(q_2 + q_3)$$

$$z = q_1 + 0.7$$

تحلیل نتایج و تایید صحت عملکرد ربات

خروجی های به دست آمده تطابق کاملی با ساختار فیزیکی ربات PRR دارند. اولاً، در معادله محور z مشاهده می کنیم که ارتفاع ربات ($z = D_0 + q_1$) کاملاً مستقل از زوایای q_2 و q_3 است. این موضوع نشان دهنده عملکرد صحیح مفصل کشویی (Prismatic) است که وظیفه دارد کل ساختار بازوها را در راستای عمودی جابه جا کند. ثانیاً، معادلات x و y دقیقاً مشابه معادلات سینماتیکی یک ربات صفحه ای دو لینکه (Planar 2R) هستند. مفصل دوم با زاویه q_2 بازوی L_2 را در صفحه XY می چرخاند و مفصل سوم با زاویه نسبی q_3 باعث می شود بازوی L_3 با زاویه مطلق $(q_2 + q_3)$ نسبت به افق قرار گیرد. عدم وجود متغیر q_1 در معادلات x و y و همچنین عدم وجود q_2 و q_3 در معادله z ، نشان دهنده استقلال حرکت عمودی از حرکات صفحه ای (ویژگی بارز ربات های SCARA و PRR مشابه آن) است. در نتیجه، این معادلات رفتار واقعی ربات را به درستی مدل کرده و صحت مقادیر خروجی کاملاً تایید می شود.

۳.۲.۲ تسک B.3 — پیاده سازی عددی و اعتبارسنجی با Numpy

توضیح پیاده سازی و کد پایتون

در این بخش، هدف اعتبارسنجی فرم بسته (Closed-form) به دست آمده برای سینماتیک مستقیم ربات است. برای این منظور، تابع پایتونی $fk(q_1, q_2, q_3)$ پیاده سازی شده است که معادلات موقعیت (مختصات x, y, z) مجری نهایی را با استفاده از توابع مثلثاتی کتابخانه NumPy محاسبه کرده و به صورت یک آرایه برمی گرداند. دقت کنید که در این تابع، از بسط روابط مثلثاتی (مانند $\cos(q_2 + q_3)$) استفاده شده است. همچنین برای مقایسه، تابع `fk_matrix_method` توسعه داده شده است که همان ماتریس های تبدیل دناویت-هارتبرگ را به صورت عددی ضرب کرده و مختصات نهایی را استخراج می کند. در نهایت، برای بررسی صحت کار، یک حلقه ۶ تکراره ایجاد شده که مقادیر مفصلی تصادفی (q_1, q_2, q_3) را در بازه های مجاز ربات تولید می کند. سپس خروجی هر دو روش محاسبه شده و خطای بین آن ها از طریق نرم اقلیدسی (Euclidean Norm) اندازه گیری می شود. کد پیاده سازی شده به شرح زیر است:

```

1  ^^Iimport numpy as np
2  ^^I
3  ^^I# Robot link parameters
4  ^^ID0, L2, L3 = 0.70, 0.40, 0.30
5  ^^I
6  ^^I# 1. Closed-form solution for forward kinematics
7  ^^Idef fk(q1, q2, q3):
8  ^^I    """Calculates the end-effector position using the closed-form equations."""
9  ^^I    Ix = np.cos(q2) * (L2 + L3 * np.cos(q3)) - L3 * np.sin(q2) * np.sin(q3)
10 ^^I    Iy = np.sin(q2) * (L2 + L3 * np.cos(q3)) + L3 * np.cos(q2) * np.sin(q3)
11 ^^I    Iz = D0 + q1
12 ^^I    return np.array([x, y, z])
13 ^^I
14 ^^I# 2. Helper function to construct numerical DH matrix
15 ^^Idef dh_matrix_num(theta, d, a, alpha):
16 ^^I    """Constructs a 4x4 DH transformation matrix."""
17 ^^I    return np.array([
18 ^^I        [np.cos(theta), -np.sin(theta)*np.cos(alpha), np.sin(theta)*np.sin(alpha), a*np.cos(theta)],
19 ^^I        [np.sin(theta), np.cos(theta)*np.cos(alpha), -np.cos(theta)*np.sin(alpha), a*np.sin(theta)],
20 ^^I        [0, np.sin(alpha), np.cos(alpha), d],
21 ^^I        [0, 0, 0, 1]
22 ^^I    ])
23 ^^I
24 ^^I# 3. Matrix method for verification
25 ^^Idef fk_matrix_method(q1, q2, q3):
26 ^^I    """Calculates end-effector position using sequential DH matrix multiplication."""
27 ^^I    IT1 = dh_matrix_num(0, D0 + q1, 0, 0)
28 ^^I    IT2 = dh_matrix_num(q2, 0, L2, 0)
29 ^^I    IT3 = dh_matrix_num(q3, 0, L3, 0)

```



```

۳۰  ^^IT_final = T1 @ T2 @ T3
۳۱  ^^Ireturn T_final[:3, 3]  # Extracting position [x, y, z]
۳۲  ^^I
۳۳  ^^I# 4. Validation process with random joint states
۳۴  ^^Inp.random.seed(42)  # Ensure reproducibility
۳۵  ^^Iprint(f"{'Test Case':<10} | {'Error (Euclidean Norm)':<25}")
۳۶  ^^Iprint("-" * 38)
۳۷  ^^I
۳۸  ^^Ifor i in range(1, 7):
۳۹  ^^I# Generating random joint values within specified ranges
۴۰  ^^Iq1_rand = np.random.uniform(0, 0.5)
۴۱  ^^Iq2_rand = np.random.uniform(-np.pi, np.pi)
۴۲  ^^Iq3_rand = np.random.uniform(-np.pi, np.pi)
۴۳  ^^I
۴۴  ^^I# Computing position via both methods
۴۵  ^^Ipos_closed = fk(q1_rand, q2_rand, q3_rand)
۴۶  ^^Ipos_matrix = fk_matrix_method(q1_rand, q2_rand, q3_rand)
۴۷  ^^I
۴۸  ^^I# Calculating the norm of the difference
۴۹  ^^Ierror = np.linalg.norm(pos_closed - pos_matrix)
۵۰  ^^I
۵۱  ^^I# Printing formatted results
۵۲  ^^Iprint(f"{i:<10} | {error:.3e}")^^I

```

نتایج اجرای کد

پس از اجرای کد بالا برای ۶ حالت مفصلی تصادفی، خروجی‌های زیر برای خطای بین دو روش به دست آمد:

Test Case	Error (Euclidean Norm)

1	1.110e-16
2	5.551e-17
3	0.000e+00
4	4.163e-17
5	5.551e-17
6	1.241e-16

تحلیل نتایج

نتایج به دست آمده به وضوح نشان دهنده هم ارزی کامل دو روش (فرم بسته ریاضی و ضرب ماتریسی عددی) است. همان طور که در خروجی مشاهده می شود، میزان خطا (نرم اقلیدسی اختلاف مختصات) در تمامی ۶ حالت آزمایش شده در مرتبه 10^{-16} یا 10^{-17} قرار دارد (و در یک مورد دقیقاً صفر است). این مقادیر بسیار کوچک در محاسبات کامپیوتری و اعداد ممیز شناور ۶۴ بیتی (Float64) معادل «صفر ماشین» (Machine Epsilon) هستند. بروز این خطای ناچیز تنها به دلیل تفاوت در نحوه گرد کردن اعداد و تاخیر عملیات در پردازنده در دو روش مختلف رخ داده است و در عمل به معنای صفر بودن خطا است. بنابراین، بر اساس این تست جامع، صحت معادلات فرم بسته برای سینماتیک مستقیم تایید می شود و دو پیاده سازی کاملاً بر هم منطبق می باشند.

۴.۲.۲ تسک B.4 — رسم فضای کاری ربات (Reachable Workspace)

معرفی تسک و هدف

در این بخش (تسک B.۴)، هدف مصورسازی فضای کاری قابل دسترسی (Reachable Workspace) برای ربات است. فضای کاری شامل تمام نقاطی در فضای سه بعدی است که مجری نهایی (End-Effector) ربات می تواند با تغییر متغیرهای مفصلی خود (q_1, q_2, q_3) به آن ها دسترسی پیدا کند. برای این منظور، از فضای حالت مفاصل نمونه برداری شده و مختصات فضایی هر نمونه با استفاده از سینماتیک مستقیم محاسبه و رسم شده است.

توضیح کد پایتون پیاده سازی شده

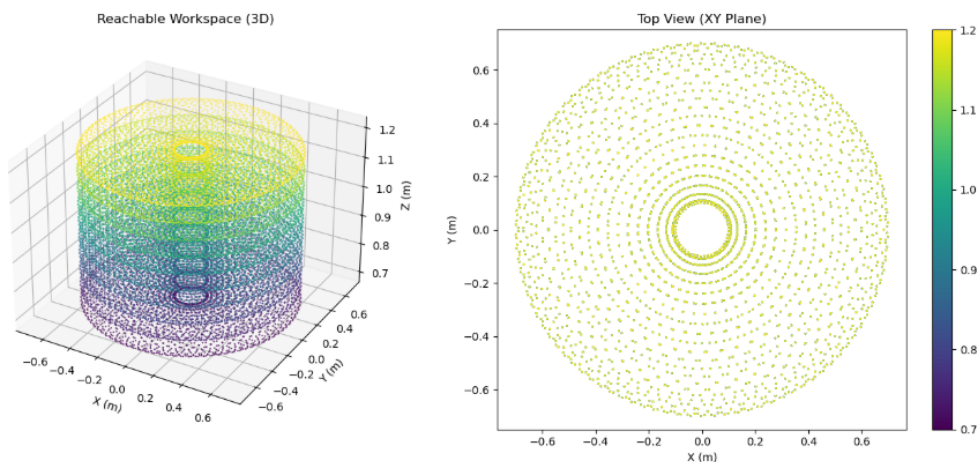
کد نوشته شده برای این بخش، ابتدا پارامترهای فیزیکی ربات ($D_0 = 0.7, L_2 = 0.4, L_3 = 0.3$) را تعریف می کند. سپس برای تولید نقاط، به جای استفاده از سه حلقه تو در تو که از نظر محاسباتی کند است، از تابع `np.meshgrid` برای ایجاد یک شبکه دوبعدی از مقادیر q_2 و q_3 استفاده شده است. متغیر q_1 (مفصل کشویی) نیز در یک حلقه جداگانه پیمایش می شود. درون حلقه، معادلات فرم بسته سینماتیک مستقیم (که در بخش های قبل به دست آمده اند) برای محاسبه ماتریس های x, y و z به کار می روند. مختصات محاسبه شده با استفاده از `flatten` به یک آرایه یک بعدی تبدیل شده و در لیست های کلی ذخیره می گردند. در نهایت، با استفاده از کتابخانه `matplotlib`، دو نمودار رسم می شود: اولی یک نمودار سه بعدی (Scatter 3D) که نقاط را در فضا نشان می دهد و دومی یک تصویر دوبعدی از نمای بالا (صفحه XY). برای درک بهتر، رنگ نقاط بر اساس ارتفاع آن ها (مقدار z) با طیف رنگی `viridis` تنظیم شده است.

```

1
2  ^^Iimport numpy as np
3  ^^Iimport matplotlib.pyplot as plt
4  ^^I
5  ^^I# 1. Define physical parameters
6  ^^IDO, L2, L3 = 0.70, 0.40, 0.30
7  ^^I
8  ^^I# 2. Create grid for joint variables (Sampling)
9  ^^I# Using meshgrid for efficient computation instead of nested loops
10 ^^In = 50 # Increased resolution for smoother visualization
11 ^^Iq1_vals = np.linspace(0, 0.5, 10)
12 ^^Iq2_vals = np.linspace(-np.pi, np.pi, n)
13 ^^Iq3_vals = np.linspace(-np.pi, np.pi, n)
    
```



```
14  ^^I
15  ^^IQ2, Q3 = np.meshgrid(q2_vals, q3_vals)
16  ^^I
17  ^^I# 3. Calculate spatial coordinates for the entire grid
18  ^^Ix_pts, y_pts, z_pts = [], [], []
19  ^^I
20  ^^Ifor q1 in q1_vals:
21  ^^I# Closed-form kinematic equations
22  ^^Ix = np.cos(Q2) * (L2 + L3 * np.cos(Q3)) - L3 * np.sin(Q2) * np.sin(Q3)
23  ^^Iy = np.sin(Q2) * (L2 + L3 * np.cos(Q3)) + L3 * np.cos(Q2) * np.sin(Q3)
24  ^^Iz = np.full_like(x, D0 + q1)
25  ^^I
26  ^^I# Flattening the arrays to append to our main list
27  ^^Ix_pts.extend(x.flatten())
28  ^^Iy_pts.extend(y.flatten())
29  ^^Iz_pts.extend(z.flatten())
30  ^^I
31  ^^I# 4. Plotting the results
32  ^^Ifig = plt.figure(figsize=(14, 6))
33  ^^I
34  ^^I# 3D Workspace plot
35  ^^Iax1 = fig.add_subplot(121, projection='3d')
36  ^^Isc = ax1.scatter(x_pts, y_pts, z_pts, c=z_pts, cmap='viridis', s=0.5)
37  ^^Iax1.set_title("Reachable Workspace (3D)")
38  ^^Iax1.set_xlabel("X (m)")
39  ^^Iax1.set_ylabel("Y (m)")
40  ^^Iax1.set_zlabel("Z (m)")
41  ^^I
42  ^^I# Top view (XY Plane) plot
43  ^^Iax2 = fig.add_subplot(122)
44  ^^Isc2 = ax2.scatter(x_pts, y_pts, c=z_pts, cmap='viridis', s=0.5)
45  ^^Iax2.set_title("Top View (XY Plane)")
46  ^^Iax2.set_xlabel("X (m)")
47  ^^Iax2.set_ylabel("Y (m)")
48  ^^Iax2.axis('equal') # Ensure aspect ratio is equal for accurate shape representation
49  ^^Iplt.colorbar(sc2, label='Height (z)')
50  ^^I
51  ^^Iplt.tight_layout()
52  ^^Iplt.show()
```



شکل ۷: نمودارهای فضای کاری ربات در حالت سه بعدی و نمای بالا

تحلیل و تفسیر نمودار سه بعدی فضای کاری

تصویر سمت چپ در خروجی ((Reachable Workspace (3D))، شکل کلی فضای کاری ربات را به صورت یک استوانه توخالی (لوله ای ضخیم) نشان می دهد. رنگ بندی این نقاط بر اساس محور Z (ارتفاع) صورت گرفته است. همان طور که نوار رنگ (Colorbar) نشان می دهد، پایین ترین نقاط (رنگ بنفش تیره) مربوط به ارتفاع 0.7 متر هستند که معادل حالتی است که مفصل کشویی اول بسته است ($q_1 = 0 \Rightarrow z = D_0 = 0.7$). بالاترین نقاط (رنگ زرد) مربوط به ارتفاع 1.2 متر هستند که معادل باز شدن کامل مفصل کشویی است ($q_1 = 0.5 \Rightarrow z = 0.7 + 0.5 = 1.2$). بنابراین، حرکت مفصل اول (q_1) صرفاً این فضای کاری را در امتداد محور Z جابجا می کند و استوانه را شکل می دهد.

تحلیل دقیق نمای بالا (صفحه XY) و مرزهای فضای کاری

تصویر سمت راست (Top View) تصویر فضای کاری روی صفحه XY را نشان می دهد که دقیقاً شبیه به یک «دونات» (حلقه) است. این شکل حاصل دوران مفاصل لولایی دوم و سوم (q_2 و q_3) است. دو مرز بسیار مهم در این نمودار قابل مشاهده و تحلیل ریاضی هستند:

- **مرز خارجی (حداکثر دسترسی):** این مرز زمانی شکل می گیرد که بازوهای دوم و سوم کاملاً در یک راستا و کشیده باشند؛ یعنی زاویه مفصل سوم صفر باشد ($q_3 = 0$). در این حالت، شعاع دسترسی ربات برابر با مجموع طول دو بازو است. با توجه به مقادیر داده شده، شعاع خارجی برابر است با $R_{max} = L_2 + L_3 = 0.4 + 0.3 = 0.7$ متر. اگر به محورهای مختصات X و Y در تصویر سمت راست دقت کنید، مشخص است که دورترین نقاط دقیقاً روی دایره ای به شعاع 0.7 قرار گرفته اند (از -0.7 تا +0.7).
- **مرز داخلی (حفره وسط فضای کاری):** این بخش، ناحیه ای در اطراف پایه ربات است که نوک ابزار هرگز نمی تواند به آن برسد. این مرز زمانی شکل می گیرد که لینک سوم کاملاً به سمت داخل تا شود؛ یعنی زاویه مفصل سوم $q_3 = 180^\circ$ یا π رادیان باشد. در این حالت شعاع دسترسی به کمترین مقدار خود می رسد که برابر با قدر مطلق تفاضل طول دو بازو است: $R_{min} = |L_2 - L_3| = |0.4 - 0.3| = 0.1$ متر. این اختلاف دقیقاً شعاع سوراخ وسط دونات را تشکیل می دهد. در نمودار سمت راست، به وضوح یک فضای خالی دایره ای در مرکز به شعاع 0.1 متر دیده می شود.

بنابراین، این نمودارها با دقت بالایی رفتار سینماتیکی ربات را تایید کرده و نشان می‌دهند که ربات در صفحه افقی تنها می‌تواند در یک نوار حلقوی بین شعاع 0.1 تا 0.7 متری خود عملیات انجام دهد.

۵.۲.۲ تسک B.5 — رسیدن به قطعات با روش جستجوی سینماتیک مستقیم

شرح پیاده‌سازی و روش کار

در این تسک، هدف قرار دادن مجری نهایی ربات در بالای سر دیسک‌های شناسایی شده در بخش بینایی ماشین است. این کار طی دو مرحله انجام می‌شود. مرحله اول تنظیم ارتفاع ربات است؛ با توجه به اینکه ارتفاع ایمن بالای میز $z_{safe} = 0.05$ متر در نظر گرفته شده و رابطه ارتفاع ربات $z_E = D_0 + q_1$ است، با یک محاسبه ساده جبری می‌توان مقدار مفصل کشویی را به صورت $q_1 = z_{safe} - D_0$ به دست آورد و ارتفاع را تنظیم کرد. مرحله دوم، یافتن زوایای مناسب برای مفاصل دورانی (q_2 و q_3) است تا ربات در مختصات صفحه‌ای (x, y) مورد نظر قرار گیرد. از آنجایی که در این بخش استفاده از سینماتیک معکوس مجاز نیست، از روش جستجوی جامع شبکه (Brute-force grid search) روی معادلات سینماتیک مستقیم استفاده شده است. در این روش، بازه حرکتی هر دو مفصل دورانی (از $-\pi$ تا π) به ۲۵۰ نقطه تقسیم شده است. الگوریتم تمام ترکیبات ممکن برای زوایا را در معادلات سینماتیک مستقیم قرار داده و خطای اقلیدسی (فاصله) بین مختصات حاصل و مرکز هدف را محاسبه می‌کند. در نهایت، زوج زاویه‌ای که کمترین خطا را تولید کند به عنوان جواب برگردانده می‌شود.

کد پایتون پیاده‌سازی شده

کد زیر پیاده‌سازی روش جستجوی جامع شبکه برای یافتن زوایای مناسب مفاصل و بررسی میزان خطای نهایی را نشان می‌دهد:

```

1  ^^Iimport numpy as np
2  ^^I
3  ^^I# Robot physical parameters
4  ^^IDO, L2, L3 = 0.70, 0.40, 0.30
5  ^^I
6  ^^I# Data of detected disks from computer vision (Section A)
7  ^^Imy_data = [
8  ^^I{'color': 'red', 'xy': (0.186, -0.104)},
9  ^^I{'color': 'red', 'xy': (0.103, 0.052)},
10 ^^I{'color': 'green', 'xy': (-0.053, 0.208)},
11 ^^I{'color': 'green', 'xy': (-0.156, 0.072)},
12 ^^I{'color': 'blue', 'xy': (-0.229, -0.186)}
13 ^^I]
14 ^^I
15 ^^Idef find_best_ik(tx, ty, L2, L3):
16 ^^I    """Finds optimal q2 and q3 using a grid search (Brute-force)."""
17 ^^I    best_err = float('inf')
18 ^^I    best_q = (0, 0)
19 ^^I

```



```

۲۰  ^^I# Sampling range for joint angles
۲۱  ^^Iq2_samples = np.linspace(-np.pi, np.pi, 250)
۲۲  ^^Iq3_samples = np.linspace(-np.pi, np.pi, 250)
۲۳  ^^I
۲۴  ^^Ifor q2 in q2_samples:
۲۵  ^^Ifor q3 in q3_samples:
۲۶  ^^I# Forward kinematics for the planar 2R part
۲۷  ^^Ix = L2 * np.cos(q2) + L3 * np.cos(q2 + q3)
۲۸  ^^Iy = L2 * np.sin(q2) + L3 * np.sin(q2 + q3)
۲۹  ^^I
۳۰  ^^I# Euclidean distance error
۳۱  ^^Idist = np.sqrt((x - tx)**2 + (y - ty)**2)
۳۲  ^^I
۳۳  ^^Iif dist < best_err:
۳۴  ^^Ibest_err = dist
۳۵  ^^Ibest_q = (q2, q3)
۳۶  ^^I
۳۷  ^^Ireturn best_q, best_err
۳۸  ^^I
۳۹  ^^I# Execution and Result Formatting
۴۰  ^^Iprint(f"{'Color':<8} | {'Coordinates (m)':<18} | {'Error (m)':<10} | {'Status'}")
۴۱  ^^Iprint("-" * 55)
۴۲  ^^I
۴۳  ^^Ifor disk in my_data:
۴۴  ^^Itx, ty = disk['xy']
۴۵  ^^I
۴۶  ^^I# Perform IK search
۴۷  ^^Iq_sol, err = find_best_ik(tx, ty, L2, L3)
۴۸  ^^I
۴۹  ^^I# Check if error is within the 5mm threshold
۵۰  ^^Ireachable = err < 0.005
۵۱  ^^Istatus = "Success" if reachable else "Out of Range"
۵۲  ^^I
۵۳  ^^Iprint(f"{disk['color']:<8} | ({tx:>6.3f}, {ty:>6.3f}) | {err:.5f} | {status}")

```

نتایج و تحلیل بررسی خطاها
با اجرای کد فوق، خروجی زیر برای ۵ دیسک هدف حاصل شد:

Color	Coordinates (m)	Error (m)	Status
-------	-----------------	-----------	--------

red	(0.186, -0.104) 0.00163	Success
red	(0.103, 0.052) 0.00226	Success
green	(-0.053, 0.208) 0.00106	Success
green	(-0.156, 0.072) 0.00067	Success
blue	(-0.229, -0.186) 0.00352	Success

با بررسی جدول خروجی، به وضوح مشاهده می‌شود که ستون خطای موقعیت (Error) برای تمام دیسک‌ها مقداری در محدوده 0.00067 تا 0.00352 متر (معادل 0.67 تا 3.52 میلی‌متر) دارد. از آنجایی که حد آستانه مجاز برای خطا در صورت سوال برابر با ۵ میلی‌متر (0.005 متر) تعیین شده بود، تمامی خطاهای به‌دست‌آمده کاملاً زیر این حد مجاز قرار دارند. بیشترین خطا مربوط به دیسک آبی‌رنگ با مقدار 3.52 میلی‌متر است که همچنان به شکل قابل‌توجهی کمتر از ۵ میلی‌متر است. این نتایج تایید می‌کنند که الگوریتم جستجوی جامع شبکه‌ای، با رزولوشن انتخاب‌شده برای نمونه‌برداری (۲۵۰ نمونه در هر زاویه)، عملکرد بسیار خوبی داشته و توانسته است ربات را با خطای بسیار ناچیز و قابل‌قبولی به دقت در بالای سر تمامی دیسک‌ها قرار دهد (وضعیت Success برای همه اهداف).

۳.۲ بخش D — سوالات تحلیلی

۱.۳.۲ پاسخ سوال اول

از آنجا که مفصل اول ربات PRR از نوع کشویی است و تنها ارتفاع ربات را در راستای محور z تغییر می‌دهد، شعاع دسترسی ربات روی سطح میز توسط دو مفصل دورانی بعدی (q_2 و q_3) و طول لینک‌های آن‌ها (L_2 و L_3) تعیین می‌شود. مفهوم کلیدی در اینجا نامساوی مثلث (Triangle Inequality) در هندسه اقلیدسی است که مرزهای داخلی و خارجی این فضای کاری را شکل می‌دهد.

تصویر حرکت مفصل دوم و سوم ربات روی صفحه افقی xy ، معادل یک مکانیزم بازوی صفحه‌ای دو لینکی (Planar Two-Link Manipulator) است. اگر مبدا مختصات (پایه ربات) را نقطه اول، مفصل سوم (آرنج) را نقطه دوم، و افکتور نهایی (نوک ابزار) را نقطه سوم در نظر بگیریم، این سه نقطه رئوس یک مثلث را در صفحه افقی تشکیل می‌دهند. طول دو ضلع این مثلث ثابت و برابر با L_2 و L_3 است.

بر اساس قضیه نامساوی مثلث در هندسه، طول ضلع سوم (یعنی فاصله مستقیم افکتور نهایی از مبدا مختصات که آن را با r نشان می‌دهیم) بین قدر مطلق تفاضل و مجموع طول دو ضلع دیگر قرار دارد:

$$|L_2 - L_3| \leq r \leq L_2 + L_3$$

این رابطه ریاضی نشان می‌دهد که فضای کاری ربات روی یک صفحه دوبعدی، یک دایره توپر نیست، بلکه یک ناحیه حلقوی یا دونات شکل است.

مرز خارجی حلقه ($L_2 + L_3$): زمانی به دست می‌آید که بازوی ربات کاملاً باز و کشیده باشد. در این حالت، زاویه مفصل سوم $q_3 = 0$ بوده و دو لینک در یک امتداد قرار می‌گیرند. در نتیجه ابزار ربات به بیشترین فاصله از پایه می‌رسد.

مرز داخلی حلقه ($|L_2 - L_3|$): زمانی به دست می‌آید که مفصل سوم کاملاً به سمت داخل تا شود ($q_3 = \pi$ یا $q_3 = -\pi$). در این حالت، لینک سوم روی لینک دوم برمی‌گردد و افکتور نهایی در نزدیک‌ترین فاصله نسبت به پایه قرار می‌گیرد.

ارتباط با ابعاد میز و ناحیه پوشش داده شده

دلیل اینکه ابعاد میز تحت پوشش ربات با عبارت $|L_2 - L_3|$ مقیاس دهی شده و به آن بستگی دارد، وجود یک ناحیه کور در مرکز میز کار است.

محدوده به شعاع $|L_2 - L_3|$ در مرکز مختصات، یک استوانه غیرقابل دسترس را تشکیل می دهد. اگر ربات در مرکز یک میز کار نصب شده باشد، نمی تواند قطعاتی که در دایره ای به شعاع $|L_2 - L_3|$ از مرکز قرار دارند را بازرسی کند. بنابراین، اگر قرار باشد ربات تمام سطح یک میز مشخص را پوشش دهد، آن میز نباید شامل نقطه ای در داخل دایره $|L_2 - L_3|$ باشد و در عین حال نباید از مرز $L_2 + L_3$ فراتر برود. اگر می خواستیم ربات نقطه کوری در مرکز نداشته باشد، از نظر هندسی باید طول دو لینک برابر در نظر گرفته می شد (یعنی $L_2 = L_3$ که نتیجه آن $|L_2 - L_3| = 0$ است). از آنجا که در این مسئله L_2 و L_3 متفاوت هستند، هندسه و شعاع داخلی منطقه ای که ربات توانایی پوشش آن را دارد، به عبارت تفاضل این دو طول مقیاس شده و مرتبط است.

۲.۳.۲ پاسخ سوال دوم

برای کاهش هزینه جستجو در الگوریتم با پیچیدگی زمانی $\mathcal{O}(N^2)$ می توان از روش Precomputed Lookup Table استفاده کرد. ایده اصلی این است که به جای حل معادلات سینماتیک معکوس برای هر هدف به صورت جداگانه یک بار پیش پردازش روی فضای پیکربندی (configuration space) ربات انجام دهیم.

جدول جستجوی پیش محاسبه شده

برای بازوی PRR، فضای پیکربندی شامل سه متغیر مفصلی است:

$$\mathbf{q} = (d_1, \theta_2, \theta_3)$$

که d_1 جابه جایی کشویی و θ_2, θ_3 زوایای مفاصل دورانی هستند.

در مرحله اول از پیش پردازش، فضای پیکربندی را با گام های گسسته نمونه برداری می کنیم. سپس به ازای هر پیکربندی $\mathbf{q}_i$ ، سینماتیک مستقیم (forward kinematics) را محاسبه و موقعیت نوک ابزار $\mathbf{p}_i = f(\mathbf{q}_i)$ را به دست می آوریم:

$$\mathbf{p}_i = f(\mathbf{q}_i) \in \mathbb{R}^2, \quad i = 1, 2, \dots, N$$

در مرحله دوم، جدولی به شکل $\{(\mathbf{p}_i, \mathbf{q}_i)\}$ می سازیم و آن را در یک ساختار k-d tree یا hash grid ذخیره می کنیم تا جستجوی نزدیک ترین همسایه (nearest neighbor search) با پیچیدگی $\mathcal{O}(\log N)$ ممکن شود.

در مرحله سوم، برای هر هدف جدید $\mathbf{p}^*$ (از جمله ۳۰ قطعه روی میز)، نزدیک ترین نقطه ای از پیش محاسبه شده را از جدول استخراج می کنیم:

$$\mathbf{q}^* = \arg \min_{\mathbf{q}_i} \|\mathbf{p}_i - \mathbf{p}^*\|_2$$

تحلیل هزینه

هزینه ی پیش پردازش یک بار انجام می شود و برابر است با:

$$T_{\text{pre}} = \mathcal{O}(N^2)$$



پس از آن، هزینه جستجو برای هر یک از $M = 30$ هدف برابر است با:

$$T_{\text{query}} = \mathcal{O}(M \log N)$$

بنابراین هزینه کل amortize شده برای همه اهداف برابر است با:

$$T_{\text{total}} = \mathcal{O}(N^2 + M \log N)$$

که در آن جمله $\mathcal{O}(N^2)$ تنها یک بار پرداخت می شود و هزینه آن بین تمام M هدف سرشکن (amortise) می گردد. میانگین هزینه هر هدف به صورت زیر کاهش می یابد:

$$\bar{T} = \mathcal{O}\left(\frac{N^2}{M} + \log N\right)$$

برای $M = 30$ ، این کاهش کاملاً محسوس است.

۳.۳.۲ پاسخ سوال سوم

۱. اعوجاج شعاعی (Radial Distortion)

شایع ترین نوع اعوجاج لنز است. در این نوع اعوجاج که شایع ترین نوع اعوجاج لنز است، نقاط تصویر در راستای شعاعی نسبت به مرکز اپتیکال جابه جا می شوند. مدل ریاضی آن به صورت زیر است:

$$x' = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

$$y' = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$$

که در آن $r^2 = x^2 + y^2$ فاصله شعاعی از مرکز تصویر و k_1, k_2, k_3 ضرایب اعوجاج شعاعی هستند. اگر $k_1 < 0$ باشد، اعوجاج بشکهای (barrel distortion) و اگر $k_1 > 0$ باشد، اعوجاج بالشتکی (pincushion distortion) رخ می دهد.

۲. اعوجاج مماسی (Tangential Distortion)

ناشی از عدم موازی بودن کامل صفحه لنز با صفحه سنسور است:

$$x' = x + [2p_1 xy + p_2(r^2 + 2x^2)]$$

$$y' = y + [p_1(r^2 + 2y^2) + 2p_2 xy]$$

که p_1, p_2 ضرایب اعوجاج مماسی هستند.

۳. سایر اعوجاج های محیط صنعتی

علاوه بر موارد فوق، در محیط های صنعتی با موارد زیر نیز مواجه هستیم:

- انحراف کروی (Spherical Aberration): شعاع های نوری حاشیه لنز در نقطه ی کانونی متفاوتی نسبت به شعاع های مرکزی تمرکز می یابند.



- انحراف رنگی (Chromatic Aberration): طول موج‌های مختلف نور در نقاط کانونی متفاوت متمرکز می‌شوند که منجر به حاشیه‌های رنگی در لبه‌های اجسام می‌گردد.

تصحیح اعوجاج لنز باید اولین مرحله‌ی پایپ‌لاین، پیش از هر پردازش دیگری، قرار گیرد. دلیل این امر آن است که اعوجاج لنز یک خطای سیستماتیک در سطح سنسور است و تمام مراحل بعدی از جمله تصحیح پرسپکتیو، آشکارسازی نشانگرها (markers) و تخمین وضعیت (pose estimation) بر فرض هندسه‌ی تصویر ایده‌آل بنا شده‌اند.

۴.۳.۲ سوال چهارم

خطای آفست سیستماتیک (Systematic Offset) به معنای یک انحراف ثابت، قابل پیش‌بینی و تکرارپذیر بین خروجی مدل ریاضی (سینماتیک مستقیم تحلیلی) و خروجی شبیه‌ساز است. اگر در ساخت فایل URDF اشتباهاتی رخ دهد که ساختار هندسی یا دستگاه‌های مختصات را نسبت به قراردادهای ریاضی (مانند پارامترهای DH) جابه‌جا کند، این نوع خطا در خروجی تابع `getLinkState` ایجاد می‌شود. مهم‌ترین دلایل هندسی و ساختاری بروز این خطا عبارتند از:

۱. عدم تطابق در بردار انتقال و دوران مفاصل (Joint Origin Misalignment): در ساختار URDF، موقعیت هر لینک نسبت به لینک قبلی توسط تگ `<origin xyz="..." rpy="...">` در داخل تعریف مفصل (`<joint>`) مشخص می‌شود. این مقادیر مستقیماً بیانگر طول لینک‌ها و زوایای انحراف مفصلی در معادلات DH هستند. اگر مقدار وارد شده در فایل URDF (مثلاً طول لینک دوم L_2) حتی چند میلی‌متر با مقادیر ثابت دستگاه معادلات تحلیلی تفاوت داشته باشد، یک خطای انتقال ثابت به تمامی لینک‌های بعدی زنجیره سینماتیکی اعمال شده و خطای آفست ایجاد می‌کند.

۲. تداخل فریم مرکز جرم با فریم سینماتیکی (CoM vs. URDF Link Frame): تابع `getLinkState` در کتابخانه PyBullet تاپلی از چندین خروجی را باز می‌گرداند. ایندکس ۰ مختصات مرکز جرم (Center of Mass Position) را در فضای جهانی نشان می‌دهد، در حالی که ایندکس ۴ مختصات دقیق مبدا دستگاه متصل به لینک (World Link Frame Position) را باز می‌گرداند. اگر در فایل URDF، درون تگ `<inertial>` مقدار `<origin>` غیر صفر تعریف شده باشد و شما به اشتباه ایندکس ۰ را با معادلات ریاضی خود مقایسه کنید، دقیقاً به اندازه بردار انتقال مرکز جرم، یک خطای سیستماتیک در تمام خروجی‌ها مشاهده خواهید کرد.

۳. اشتباه در تعریف مفصل مجازی افکتور نهایی (End-effector Dummy Link): نقطه ابزار یا افکتور نهایی در سینماتیک معمولاً به عنوان یک نقطه ریاضی در انتهای لینک آخر در نظر گرفته می‌شود. برای استخراج این نقطه در URDF، عموماً یک مفصل ثابت (Fixed Joint) بدون جرم و یک لینک مجازی تعریف می‌شود. اگر فاصله و جهت این مفصل ثابت با آنچه در ماتریس انتقال نهایی (برای مثال ماتریس T_3^0) فرض شده است، همخوانی نداشته باشد (مثلاً آفست نوک ابزار صفر فرض شده باشد اما در URDF مقداری به آن اختصاص یابد)، خطای آفست سیستماتیک در فضای دکارتی پدیدار می‌شود.

۴. جابه‌جایی مبدا لینک پایه (Base Frame Offset): اگر فایل URDF به گونه‌ای طراحی شده باشد که مبدا مختصات لینک پایه ربات با مبدا هندسی مفصل اول (یا سطح میز کار) دقیقاً منطبق نباشد، یا هنگام فراخوانی تابع بارگذاری ربات در محیط PyBullet، ربات دقیقاً در مختصات (0, 0, 0) و بدون دوران اولیه قرار نگیرد، تمام محاسبات استخراج شده توسط `getLinkState` دارای یک شیفت سراسری و سیستماتیک نسبت به مختصات جهانی محاسبات تحلیلی خواهند بود.

۵.۳.۲ سوال پنجم

در پیکربندی Eye-in-Hand، دوربین به جای یک نقطه ثابت در محیط، به صورت صلب روی یکی از لینک‌های متحرک ربات (در این مسئله، مفصل دوم) نصب می‌شود. در نتیجه، موقعیت و زاویه دید دوربین در هر لحظه به وضعیت مفاصل پیش از خود (یعنی مفصل



کشویی q_1 و مفصل دورانی q_2 بستگی دارد. برای استخراج موقعیت قطعات روی میز کار صرفاً با استفاده از سینماتیک مستقیم، پایپ لاین به شرح زیر تغییر می‌کند:

۱. محاسبه موقعیت و جهت‌گیری دوربین در فضای جهانی: ابتدا مقادیر فعلی مفاصل q_1 و q_2 خوانده می‌شود. با قرار دادن این مقادیر در معادلات سینماتیک مستقیم (Forward Kinematics)، ماتریس تبدیل همگن مفصل دوم نسبت به دستگاه پایه ربات به دست می‌آید. از آنجا که دوربین به صورت صلب به این مفصل متصل است، یک ماتریس تبدیل ثابت هندسی (آفست نصب دوربین) نیز وجود دارد. حاصل ضرب ماتریس سینماتیک مستقیم در این ماتریس ثابت، موقعیت و زاویه دقیق لنز دوربین را در دستگاه مختصات پایه ربات مشخص می‌کند.

۲. پردازش تصویر و تشکیل پرتوهای نوری: تصویر گرفته شده توسط دوربین با استفاده از الگوریتم‌های بینایی ماشین (مشابه بخش‌های قبل) پردازش می‌شود تا مختصات پیکسلی (u, v) برای مرکز هر دیسک رنگی پیدا شود. با استفاده از پارامترهای ذاتی دوربین (Intrinsic Parameters)، این مختصات دوبعدی پیکسلی به یک پرتو برداری سه‌بعدی (Ray) در دستگاه مختصات محلی دوربین (Camera Frame) نگاشت می‌شود.

۳. انتقال پرتوها به دستگاه پایه ربات: با استفاده از موقعیت دوربین که در مرحله اول از طریق سینماتیک مستقیم به دست آمد، امتداد این پرتوهای نوری از دستگاه محلی دوربین به دستگاه مختصات جهانی یا پایه ربات (Base Frame) منتقل می‌شود. اکنون ما یک خط سه‌بعدی در فضای واقعی داریم که از لنز دوربین شروع شده و از مرکز هدف عبور می‌کند.

۴. تقاطع هندسی با صفحه کار (یافتن مختصات نهایی): از اطلاعات مسئله می‌دانیم که تمام قطعات روی سطح صاف میز کار قرار دارند و سطح میز دقیقاً معادل صفحه $z = 0$ در دستگاه مختصات جهانی است. برای یافتن موقعیت دقیق قطعه، صرفاً کافی است محل برخورد پرتو سه‌بعدی منتقل شده به فضای جهانی را با صفحه $z = 0$ محاسبه کنیم. نقطه تقاطع، دقیقاً مختصات x و y قطعات را روی میز به دست می‌دهد.

به این ترتیب، کل فرآیند تبدیل پیکسل به مختصات ربات، بدون نیاز به حل سینماتیک معکوس و تنها با یک استفاده مستقیم از معادلات سینماتیک مستقیم و یک تقاطع خط و صفحه، پیاده‌سازی می‌گردد.